Tema 3

Diagramas Entidad/Relación (ER)

Base de Datos

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 4
1. Introducción 5
1.1 Origen y elementos básicos 5
1.2 Dia, una herramienta para diseñar modelos ER 6
2. Entidades y relaciones. Atributos. Cardinalidad 12
2.1 Entidades 12
2.2 Relaciones 13
2.3 Atributos 16
2.4 Cardinalidad de los atributos 16
2.5 Ejemplos 21
2.5.1 Ejemplo de un sistema de Ticketing 21
2.5.2 Ejemplo de una agencia de viajes 22
2.5.3 Ejemplo de una universidad 23
2.5.4 Ejemplo de un taller mecánico 24
3. Entidades fuertes vs entidades débiles. Relaciones de dependencia en existencia y en identificación 25
3.1 Entidad fuerte vs. Entidades débiles 25
3.2 Relaciones de dependencia en existencia de identificación 29
4. El modelo E/R ampliado. Generalizaciones 36
4.1 El modelo E/R ampliado 36
4.2 Proceso de generalización 38
4.3 Proceso de especialización 39
5. Paso del modelo E/R al modelo relacional 43
6. Normalización de modelos relacionales, formas normales y desnormalización 53
5.1 Normalización de los modelos relacionales 53
5.2 Las formas normales 53
5.2.1 Primera Forma Normal (1FN) 54
5.2.2 Segunda Forma Normal (2FN) 54
5.2.3 Tercera Forma Normal (3FN) 55

5.2.4 Forma Normal de Boyce-Codd (FNBC) 55
5.2.5 Cuarta Forma Normal (4FN) 55
5.2.6 Quinta Forma Normal (5FN) 56
5.3 Ejemplos de normalización 57
5.4 Desnormalización 64
6.Resumen 67
Glosario 69
Bibliografía 71

Objetivos de la unidad

Los objetivos de aprendizaje para las personas que concluyan el estudio de este material son la asimilación de los siguientes conceptos y la reflexión sobre algunas de las ventajas que nos proporciona:

- **Comprender las bases del diseño de bases de datos:** El modelo ER es una técnica fundamental en el diseño de bases de datos, por lo que es importante comprenderlo para poder diseñar bases de datos eficientes y precisas.
- **Identificar entidades y relaciones**: Este modelo permite identificar las entidades y relaciones relevantes en un sistema, lo que es clave para entender la estructura de los datos.
- **Definir atributos:** Al utilizar el modelo ER, es posible definir los atributos que se asociarán a cada entidad, lo que permite precisar aún más la información que se almacenará en la base de datos.
- **Analizar la semántica del sistema**: El modelo permite analizar la semántica del sistema, es decir, entender el significado de cada elemento que forma parte de este.
- **Herramienta de comunicación entre usuarios y desarrolladores:** El modelo ER se puede utilizar como una herramienta de comunicación entre los usuarios y los desarrolladores de la base de datos, lo que facilita la comprensión de los requisitos y la implementación de soluciones adecuadas.
- **Facilitar el mantenimiento y evolución de la base de datos**: El uso del modelo entidad-relación permite mantener y evolucionar la base de datos a medida que se producen cambios en el sistema, lo que resulta en una base de datos más flexible y adaptable.

Tech Hub <FP> euroformac

1. Introducción

1.1 Origen y elementos básicos

El **modelo entidad-relación (ER)** fue desarrollado en la década de 1970 por **Peter Chen** como una forma de describir la estructura lógica de una base de datos de manera clara y precisa.

Está basado en la idea de que un sistema está compuesto por:

- **Entidades**, que son objetos o conceptos del mundo real que tienen una existencia independiente
- **Relaciones**, que son asociaciones entre estas entidades.
- Cada entidad puede tener **atributos**, que son características o propiedades que describen la entidad.

Las **entidades se representan mediante rectángulos**, las **relaciones mediante líneas** y los **atributos mediante círculos o elipses**.

Las entidades y relaciones pueden tener nombres descriptivos que reflejen su significado en el sistema, y los atributos pueden tener tipos de datos específicos como texto, números o fechas.

En la imagen podemos apreciar los siguientes elementos:

- Dos entidades: Agentes comerciales y Clientes.
- Tienen una relación llamada Comercialización, en la que un agente puede gestionar varios clientes, pero un cliente solamente trata con un agente.
- La entidad Agentes Comerciales tiene los atributos nombre y porcentaje de comisión, y la entidad Clientes está descrita por los atributos razón social y límite de crédito.

El modelo ER es una herramienta importante en el diseño de bases de datos ya que permite visualizar la estructura de los datos y asegurarse de que se capturen todos los elementos necesarios para el sistema.

También resulta útil para comunicar la estructura de la base de datos a los usuarios y desarrolladores, y para adaptar la base de datos a medida que cambian las necesidades del sistema.

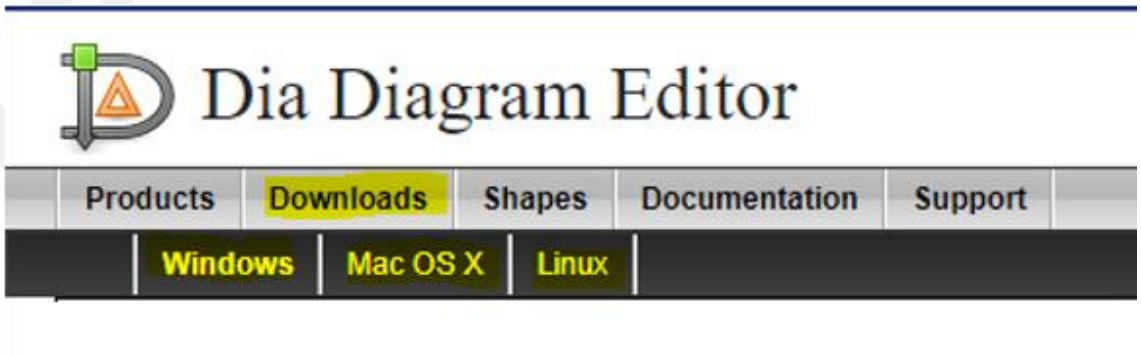
1.2 Dia, una herramienta para diseñar modelos ER

Dia es una herramienta visual que permite crear diagramas de múltiples tipologías, incluyendo diagramas **Entidad-Relación (ER)** y de **Bases de datos**.

Podemos descargarlo desde el siguiente enlace http://dia-installer.de/download/index.html

Existen versiones para Windows, MAC, y Linux.

Dependiendo de cuál sea el sistema operativo en el que se pretenda instalar, se escogerá la versión que se adecue a este, teniendo en cuenta que la versión puede ir variando a lo largo del tiempo.



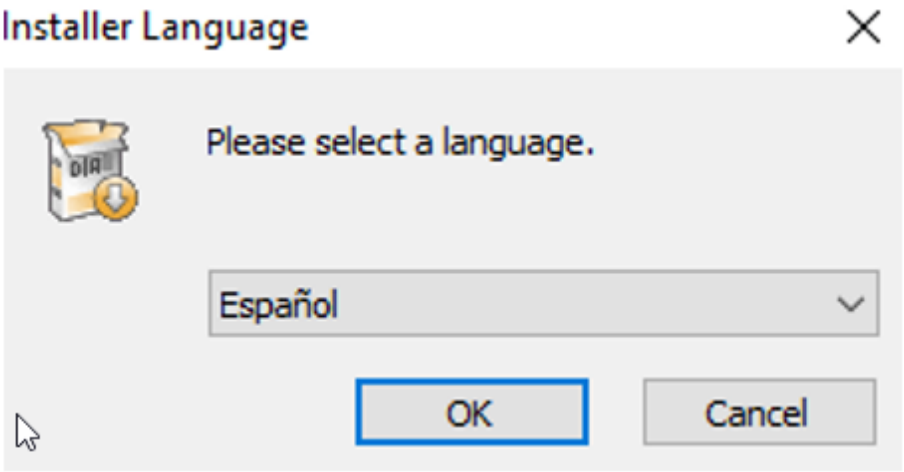
Página de descarga

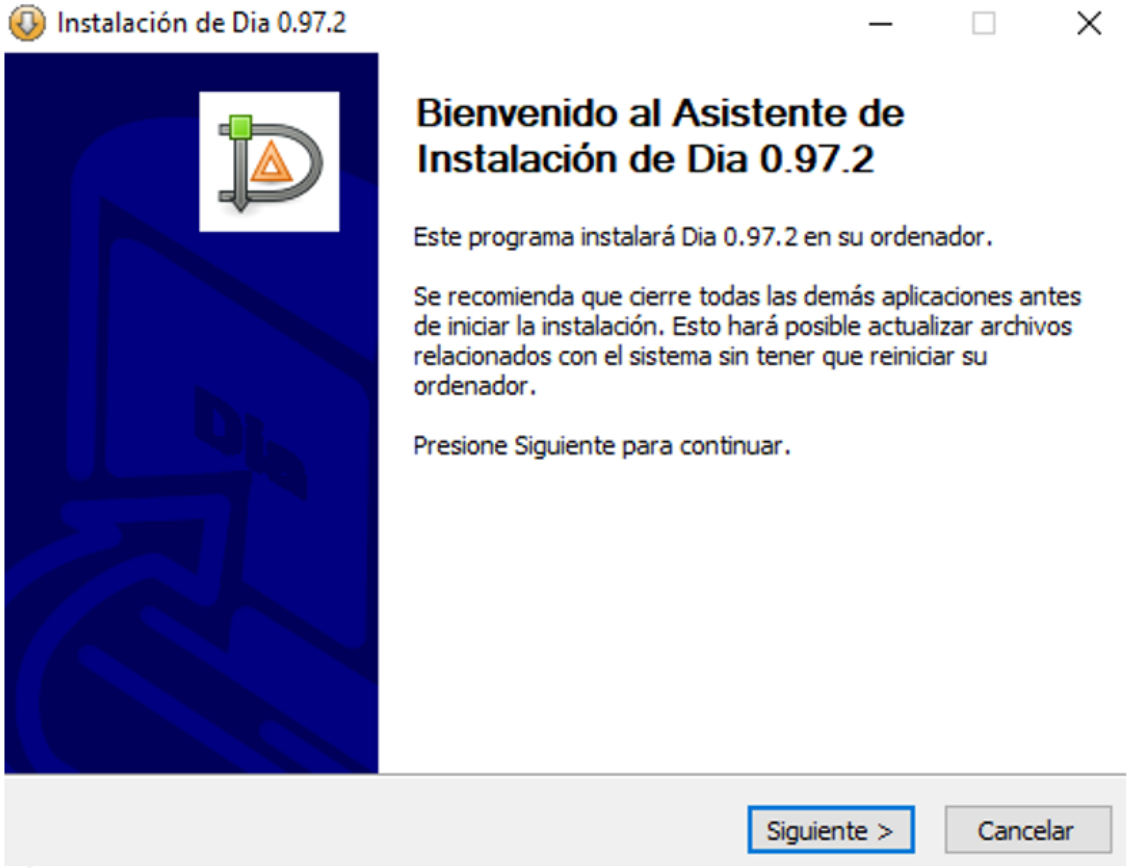
Descarga para Windows

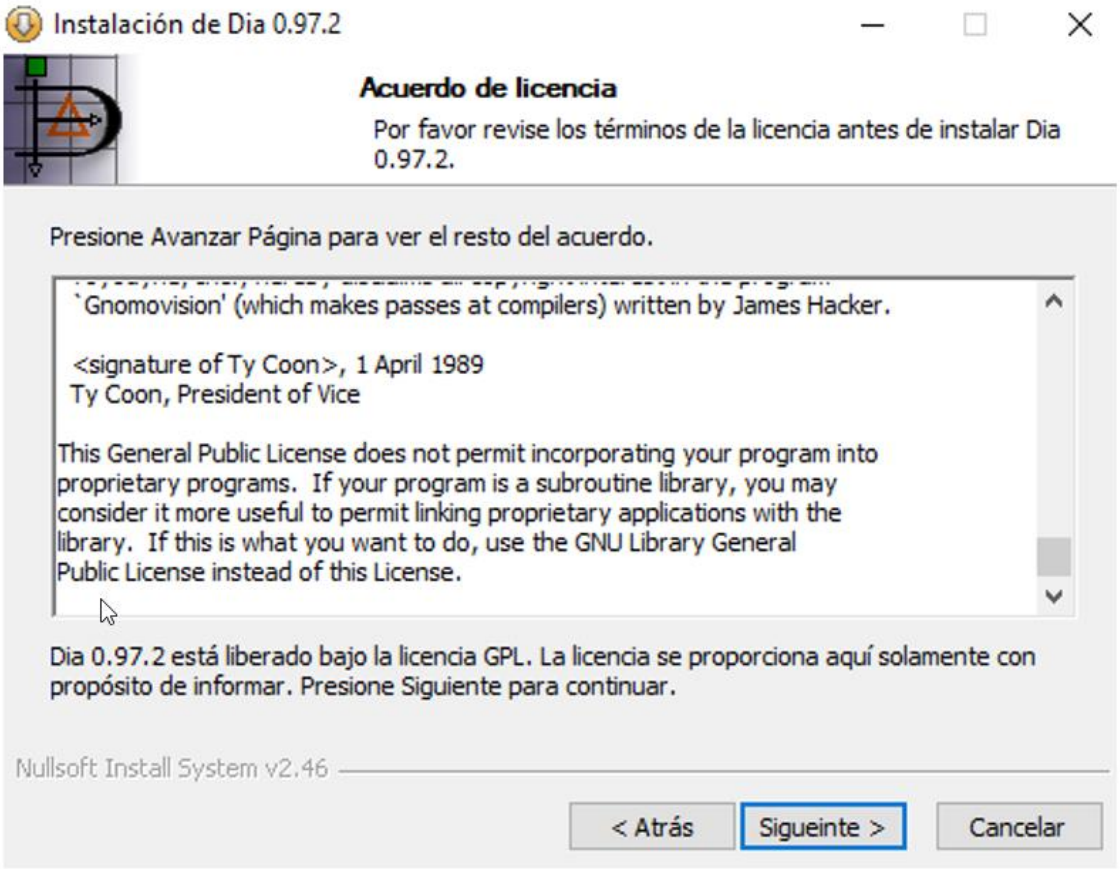
Descarga para MAC

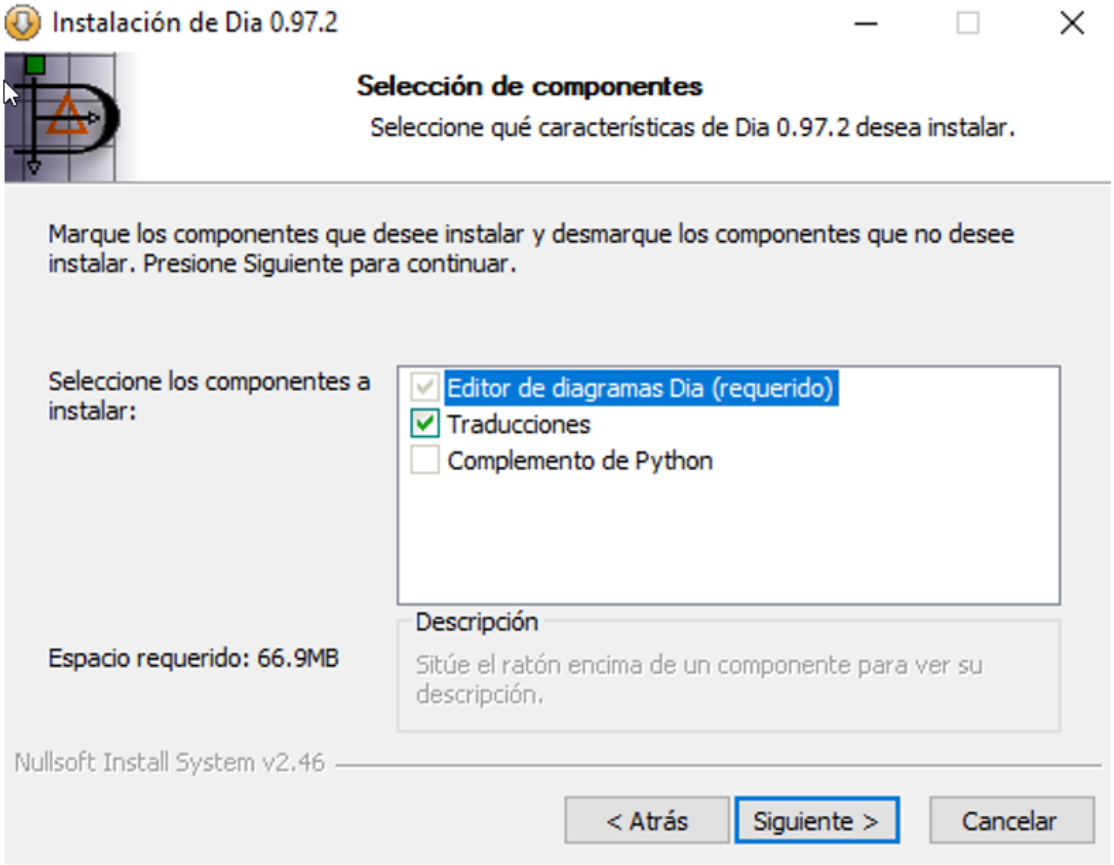
Descarga para Linux

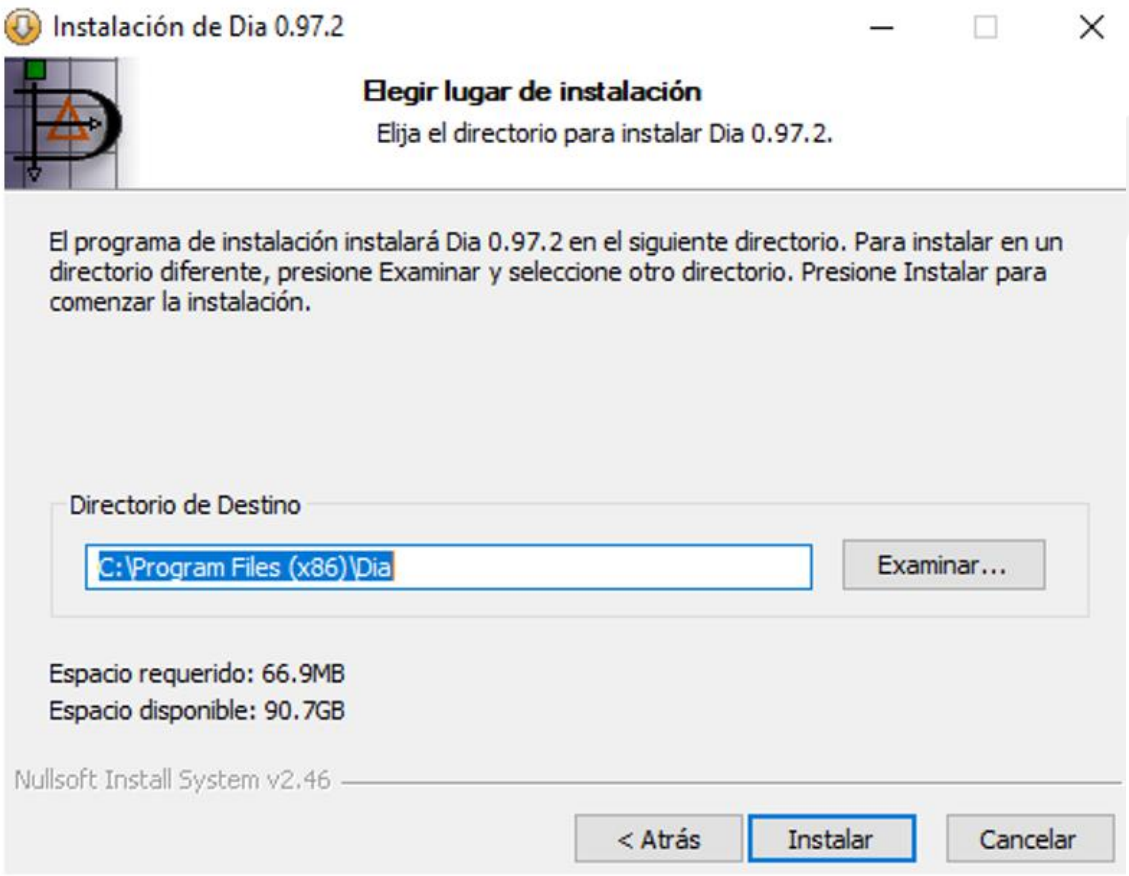
El proceso de instalación es muy rápido y sencillo, basta seguir el asistente de instalación para tenerlo disponible:


Installer Language
Please select a language.
Español
OK
Cancel

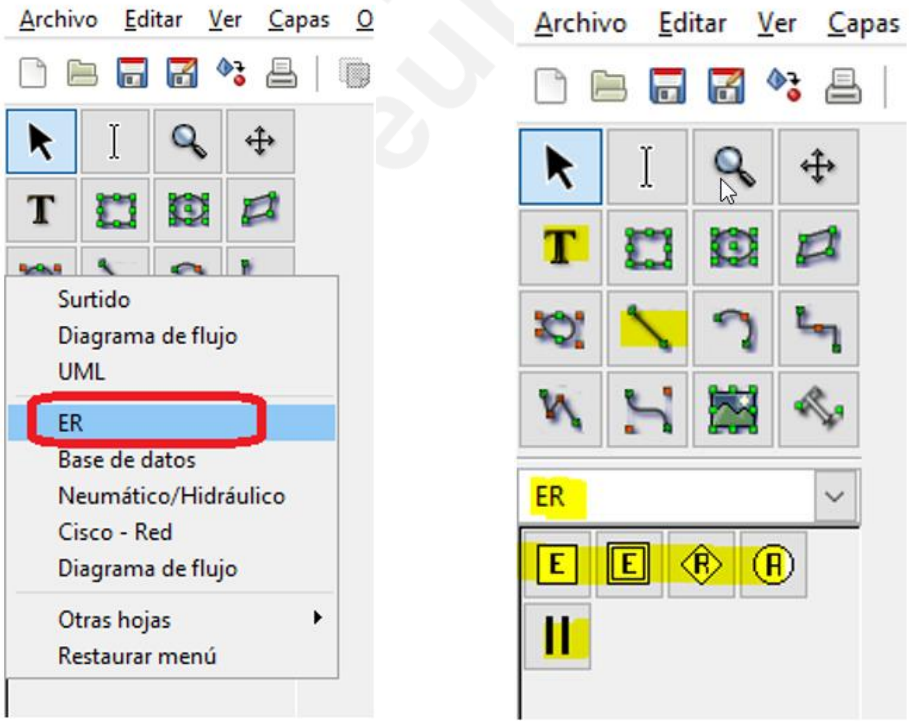

Instalación de Dia 0.97.2
Bienvenido al Asistente de Instalación de Dia 0.97.2
Este programa instalará Dia 0.97.2 en su ordenador.
Se recomienda que cierre todas las demás aplicaciones antes de iniciar la instalación. Esto hará posible actualizar archivos relacionados con el sistema sin tener que reiniciar su ordenador.
Presione Siguiente para continuar.
Siguiente >
Cancelar


Instalación de Dia 0.97.2
Acuerdo de licencia
Por favor revise los términos de la licencia antes de instalar Dia 0.97.2.
Presione Avanzar Página para ver el resto del acuerdo.
`Gnomovision' (which makes passes at compilers) written by James Hacker.
<signature of Ty Coon>, 1 April 1989
Ty Coon, President of Vice
This General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Library General Public License instead of this License.
Dia 0.97.2 está liberado bajo la licencia GPL. La licencia se proporciona aquí solamente con propósito de informar. Presione Siguiente para continuar.
Nullsoft Install System v2.46
< Atrás
Sigueinte >
Cancelar


Instalación de Dia 0.97.2
Selección de componentes
Seleccione qué características de Dia 0.97.2 desea instalar.
Marque los componentes que desee instalar y desmarque los componentes que no desee instalar. Presione Siguiente para continuar.
Seleccione los componentes a instalar:
Editor de diagramas Dia (requerido)
Traducciones
Complemento de Python
Descripción
Sitúe el ratón encima de un componente para ver su descripción.
Espacio requerido: 66.9MB
Nullsoft Install System v2.46
< Atrás
Siguiente >
Cancelar



Una vez instalado, seleccionamos el **Modelo Entidad-Relación (ER)**, y ya se puede disponer de sus componentes para poder usarlos:



2. Entidades y relaciones. Atributos. Cardinalidad

2.1 Entidades

En el modelo **entidad-relación (ER)** una entidad es un objeto o concepto del mundo real que tiene una existencia independiente y que es relevante para el sistema que se está diseñando.

Las entidades se representan mediante rectángulos en el diagrama ER y pueden tener nombres descriptivos que reflejen su significado en el sistema.

Es muy importante identificar correctamente las entidades en un sistema, ya que esto asegura que se capturen todos los elementos necesarios en la base de datos.

Cada entidad debe ser única y estar claramente definida en términos de sus atributos y relaciones con otras entidades en el sistema.

Veamos la aplicación de este modelo entidad-relación aplicado a un caso real – FestivalFest

FestivalFest es un festival de música al aire libre en el que participan varios artistas, hay diferentes escenarios y se venden entradas a los asistentes. Para que el equipo técnico pueda gestionar logísticamente todo el evento, necesitan una base de datos que refleje estas realidades: quién toca, dónde y cuándo, y qué entradas adquiere cada persona.

En tu herramienta de diagramado Dia, necesitas diseñar el diagrama ER inicial definiendo las cinco entidades principales de FestivalFest:

- Artista
- Escenario
- Concierto
- Asistente
- Entrada
- Coordinador

Para cada entidad, necesitas indicar su clave primaria.

Solución

Abre Dia y selecciona en la paleta “Modelo Entidad-Relación (ER)”.

Arrastra al lienzo cinco rectángulos; renómbralos uno a uno:

- Artista
- Escenario
- Concierto
- Asistente
- Entrada
- Coordinador

Dentro de cada rectángulo añade un texto con el nombre de la clave primaria, por ejemplo:

- ArtistaID(PK)
- EscenarioID(PK)
- ConciertoID(PK)
- AsistenteID(PK)
- EntradaID (PK)
- CoordinadorID (PK)

Ajusta el tamaño de cada entidad para que el texto quede bien centrado.

Guarda tu diagrama como archivo .dia y, opcionalmente, expórtalo a .png para incluirlo en la documentación.

Con este ejercicio tienes el conjunto de entidades de FestivalFest correctamente identificado. En el siguiente apartado añadiremos sus atributos y definiremos cómo se relacionan entre sí.

2.2 Relaciones

En el modelo **ER** las **relaciones** representan la conexión lógica entre dos o más entidades.

Las relaciones se representan mediante líneas que conectan las entidades y se etiquetan con un verbo que describe la relación entre ellas.

Las relaciones pueden ser de tres tipos:

- **Uno a uno (1:1):** Indica que una entidad se relaciona con otra entidad en una única correspondencia. Por ejemplo:
  - Un pasaporte se relaciona con un solo titular y un titular tiene solo un pasaporte.
  - Una huella dactilar se relaciona con una sola persona o una persona posee esa huella dactilar única.

- **Uno a muchos (1:N):** Indica que una entidad se relaciona con muchas entidades de otra entidad, pero cada entidad de la segunda entidad solo se relaciona con una entidad de la primera entidad. Por ejemplo:
  - Un cliente puede realizar muchas compras, pero cada compra está relacionada con un solo cliente.
  - Un agente comercial gestiona a muchos clientes, pero un cliente es gestionado únicamente por un agente comercial.
- **Muchos a muchos (N:N):** Indica que muchas entidades de una entidad se relacionan con muchas entidades de otra entidad. Por ejemplo:
  - Muchos estudiantes pueden estar matriculados en muchas asignaturas.
  - Muchos proveedores sirven una misma marca de producto, y esta a su vez, puede ser servida por muchos proveedores.

Atributos de las relaciones

Hay que tener en cuenta que las relaciones pueden tener atributos propios, como cualquier entidad independiente. Por ejemplo:

- En una relación entre un/a estudiante y una asignatura, los atributos de la relación podrían ser la nota obtenida y en qué fecha se obtuvo.
- En una relación entre proveedores y marcas, el atributo de la relación podría ser el precio al que se sirve el producto.

Siguiendo con el mismo ejemplo práctico del apartado anterior, veamos cómo se relacionen las distintas entidades entre sí:

En FestivalFest, además de las entidades Artista, Escenario, Concierto, Asistente y Entrada, necesitamos reflejar cómo se relacionan entre sí. Algunas reglas de negocio son:

- Cada escenario tiene asignado un único coordinador y un coordinador solo gestiona un escenario (relación 1:1).
- Un artista puede participar en varios conciertos, pero cada concierto lo da un solo artista (1:N).
- Cada asistente puede acudir a muchos conciertos y cada concierto puede ser visto por muchos asistentes (N:N).

Con Dia, añade al diagrama las relaciones indicadas, marcando su tipo de cardinalidad (1:1, 1:N, N:N) entre:

1. Escenario ↔ Coordinador
2. Artista ↔ Concierto
3. Asistente ↔ Concierto

Solución

1. En Dia, abre tu diagrama ER de FestivalFest y arrastra tres símbolos "Relación" al lienzo.
2. Conecta cada relación:
   - **Escenario — Coordinador**: dibuja la línea y en las propiedades de la relación pon "1" junto a Escenario y "1" junto a Coordinador (1:1).
   - **Artista — Concierto**: une Artista con Concierto y ajusta cardinalidad "1" en Artista y "N" en Concierto (1:N).
   - **Asistente — Concierto**: une Asistente y Concierto, configurando ambos extremos como "N" (N:N).

3. Etiqueta cada línea con un verbo que describa la relación:
   - "gestiona" entre Coordinador y Escenario,
   - "ofrece" entre Artista y Concierto,
   - "asiste a" entre Asistente y Concierto.

4. Guarda y exporta el diagrama.

Al finalizar tendrás un modelo ER que refleja correctamente las tres tipologías de relaciones en FestivalFest, listo para continuar con los atributos y las cardinalidades de las columnas en la siguiente sección.

2.3 Atributos

Un **atributo** se refiere a una característica o propiedad que describe una entidad en el modelo.

Los atributos se utilizan para proporcionar información adicional sobre una entidad, lo que permite a los usuarios comprender mejor su naturaleza y su papel dentro del modelo.

En general, **hay tres tipos principales de atributos** en un modelo ER:

- **Atributos simples**: Son aquellos que describen una sola característica de una entidad. Por ejemplo, un atributo simple de un cliente podría ser su nombre o su dirección.
- **Atributos compuestos**: Son aquellos que se componen de dos o más atributos simples. Por ejemplo, la dirección de un cliente podría estar compuesta por su calle, ciudad, estado y código postal.
- **Atributos derivados:** Son aquellos que se calculan a partir de otros atributos en el modelo. Por ejemplo, la edad de un cliente se puede calcular a partir de su fecha de nacimiento.

Cabe decir que, en una base de datos relacional, los atributos compuestos suelen formar parte de otras entidades relacionadas, y que los derivados no suelen formar parte de ninguna entidad, calculándose su valor cuando son requeridos.

2.4 Cardinalidad de los atributos

Hace referencia al número de valores que puede/debe admitir un atributo en concreto.

En este sentido, podemos distinguir **dos tipos de atributos:**

- **Monovalor**: Solamente admiten un valor para una entidad o relación.
- **Multivalor**: Pueden tener más de un valor para una entidad o relación.

Ejemplos de atributo multivalor

1. En una entidad Alumno podríamos definir el atributo Idiomas como multivalor, y este sería un ejemplo de los posibles valores que podría contener (separados por comas):

ID	Nombre	Idiomas
1	Eva	Español, inglés
2	María	Francés, alemán, italiano
3	Rosa	Español, ruso
4	Pedro	Inglés, italiano, chino mandarín
5	Emma	Portugués, chino

2. Imaginemos una entidad Clientes en que puede darse el caso de que tengamos varias direcciones que registrar. El diseño estructural de los atributos de la entidad pudiera ser el siguiente:
   - Nombre (monovalor)
   - Apellido (monovalor)
   - Edad (monovalor)
   - Dirección (multivalor):
     - Dirección fiscal
     - Dirección razón social
     - Dirección de entregas

La **cardinalidad de los atributos** se suele especificar agregando al nombre del atributo un paréntesis con los límites inferior y superior del número de valores que puede admitir.

- **Nombre del atributo (0, 1):** Atributo monovalor opcional (admite valores nulos).

  Por ejemplo, supongamos que tenemos una entidad "estudiante" con el siguiente conjunto de atributos:

  - Nombre
  - Apellido
  - Número de identificación
  - Número de teléfono (0, 1)

En este caso, el atributo "Número de teléfono" tiene una cardinalidad de (0, 1), lo que significa que un estudiante puede tener cero o un número de teléfono registrado.

- **Nombre del atributo (1, 1):** Atributo monovalor obligatorio (podríamos establecer un valor por defecto si el usuario no especifica otro valor).

  Por ejemplo, supongamos que tenemos una entidad "casa" con el siguiente conjunto de atributos:

  - Referencia catastral
  - Dirección (1, 1)
  - Número de habitaciones
  - Tamaño del terreno

  En este caso, el atributo "Dirección" tiene una cardinalidad de (1, 1), lo que significa que cada casa debe tener exactamente una dirección.

- **Nombre del atributo (0, n):** Atributo multivalor opcional (admite valores nulos).

  Por ejemplo, supongamos que tenemos una entidad "cliente" con el siguiente conjunto de atributos:

  - Nombre
  - Apellido
  - Correo electrónico (0, n)

  En este caso, el atributo "Correo electrónico" tiene una cardinalidad de (0, n), lo que significa que un cliente puede tener cero o más correos electrónicos registrados.

- **Nombre del atributo (1, n):** Atributo multivalor obligatorio (no admite valores nulos, debiendo especificarse al menos un valor).

  Por ejemplo, supongamos que tenemos una entidad "Libro" con el siguiente conjunto de atributos:

  - ISBN
  - Título
  - Ediciones (1, n)

En este caso, el atributo "Ediciones" tiene una cardinalidad de (1, n), lo que significa que un libro debe tener al menos una edición, pudiendo tener muchas otras con el paso del tiempo.

Continuando con el mismo ejemplo práctico veamos cómo se definen los atributos y cardinalidades – Definición de atributos y cardinalidades para "FestivalFest"

En FestivalFest, el equipo de bases de datos ya ha fijado las entidades y relaciones principales (Concierto – Artista – Asistente – Entrada – Escenario – Coordinador). Ahora toca detallar **qué datos concretos** debe guardar cada entidad y **cuántos valores** se admiten por registro (cardinalidad de los atributos). Este paso resulta decisivo: si un atributo se olvida o se marca con la cardinalidad equivocada, más tarde surgirán problemas de información duplicada o incompleta.

Cuestiones a resolver

1. Elige, para cada entidad, al menos **un atributo simple, uno compuesto, uno derivado y uno multivalor** (si procede).
2. Especifica la **cardinalidad** (0,1 – 1,1 – 0,n – 1,n) de cada atributo, justificando tu elección.
3. Explica brevemente **por qué** cada atributo se clasifica como simple, compuesto, derivado o multivalor.

Solución

Entidad	Atributo (tipo)	Cardinalidad	Justificación resumida
Artista	Nombre (simple)	(1, 1)	Identificador legible; cada artista debe tenerlo y es único.
	Contacto (*compuesto*) → Teléfono, Email	(0, 1) cada sub-atributo	Puede faltar si el mánager no lo facilita.
	Edad (*derivado*)	n/a (no se almacena)	Se calcula con AñoActual – AñoNacimiento.
	Instrumentos (*multivalor*)	(1, n)	Todo artista toca ≥ 1 instrumento; se listan todos.
Concierto	FechaHora (simple)	(1, 1)	Un concierto ocurre en un instante concreto.

	Localización *(compuesto)* → Ciudad, Recinto	(1, 1) cada sub-atributo	Lugar obligatorio dividido en dos campos.
	DuraciónMin *(derivado)*	n/a	Se obtiene restando hora fin – hora inicio.
	Géneros *(multivalor)*	(1, n)	Un mismo show puede mezclar rock, pop…
Asistente	Nombre (simple)	(1, 1)	Dato identificativo mínimo.
	Dirección *(compuesto)* → Calle, CódigoPostal, Provincia	(0, 1)	Algunos compradores prefieren solo email.
	AntigüedadAños *(derivado)*	n/a	A partir de FechaAlta.
	Aficiones *(multivalor)*	(0, n)	Se usan para marketing segmentado.
Entrada	Precio (simple)	(1, 1)	Importe final de la compra.
	QR_Detalle *(compuesto)* → CódigoQR, UrlCheckIn	(1, 1)	Información para control de accesos.
	ImporteComisión *(derivado)*	n/a	Precio × 0,05 calculado al vuelo.
	BeneficiosExtra *(multivalor)*	(0, n)	Camiseta, backstage, consumición…
Escenario	NombreEscena (simple)	(1, 1)	Identificación pública («Main Stage»).
	Sonido *(compuesto)* → PotenciaW, EmpresasPA	(1, 1) y (0, n)	Potencia obligatoria; varias empresas sonoras posibles.
	HorasUsoDiarias *(derivado)*	n/a	Se calcula sumando las franjas de conciertos asignados.
	EquipamientoEspecial *(multivalor)*	(0, n)	Lásers, pirotecnia, pantallas…
Coordinador	Apellidos (simple)	(1, 1)	Dato identificativo.

	Identificación *(compuesto)* → DNI, SSN	(1, 1) cada sub-atributo	Ambos documentos requeridos por normativa laboral.
	AñosExperiencia *(derivado)*	n/a	AñoActual – AñoInicioProfesión.
	Certificaciones *(multivalor)*	(0, n)	PRL, primeros auxilios, sonido…

Por qué se catalogan así

- **Simples**: contienen un solo dato indivisible (Nombre, Precio, FechaHora).
- **Compuestos**: agrupan datos íntimamente ligados que se descomponen en columnas hijas (Contacto, Localización, Dirección).
- **Derivados**: no se almacenan físicamente; se calculan cuando se necesitan (Edad, DuraciónMin). Mantienen la base ligera y evitan incoherencias.
- **Multivalor**: aceptan varias ocurrencias por fila (Instrumentos, Géneros). Se normalizan en tablas puente, pero aquí se declaran para mostrar la cardinalidad (1,n o 0,n).

Con esta ficha de atributos y cardinalidades, *FestivalFest* ya cuenta con la "letra pequeña" que faltaba para completar su modelo ER antes de entrar en detalles de normalización y posteriores transformaciones al modelo relacional.

2.5 Ejemplos

2.5.1 Ejemplo de un sistema de Ticketing

En un sistema de **Ticketing** las entidades podrían incluir:

- **Usuario**: Representa a las personas que utilizan el sistema de Ticketing. Podría tener atributos como el nombre, correo electrónico, teléfono, departamento o equipo al que pertenece, etc.
- **Ticket**: Representa un problema o solicitud reportado por un usuario. Podría tener atributos como el número de ticket, la fecha y hora de creación, la prioridad, el estado actual (abierto, en progreso, cerrado, etc.), la descripción del problema o solicitud, etc.
- **Agente**: Representa a los agentes o técnicos que trabajan en el sistema de Ticketing para resolver los problemas y atender las solicitudes de los usuarios. Podría tener atributos como el nombre, correo electrónico,

teléfono, departamento o equipo al que pertenece, la carga de trabajo actual, etc.

- **Comentario**: Representa los comentarios y notas asociados a un ticket. Podría tener atributos como el texto del comentario, la fecha y hora en que se agregó, el autor del comentario (ya sea un usuario o un agente), etc.
- **Categoría**: Representa las categorías o tipos de problemas o solicitudes que los usuarios pueden reportar. Podría tener atributos como el nombre de la categoría, la descripción, el departamento o equipo responsable de la categoría, etc.

Estas entidades estarían relacionadas entre sí en el diagrama ER del sistema de Ticketing. Algunas de estas relaciones podrían ser las siguientes:

- Un ticket estaría relacionado con un usuario (1-1).
- Un ticket estaría relacionado con un agente (1-1).
- Un usuario podría tener muchos tickets creados (1-N)
- Un agente podría tener muchos tickets asignados (1-N)
- Un ticket podría tener muchos comentarios asociados (1-N).

2.5.2 Ejemplo de una agencia de viajes

En una agencia de viajes podríamos considerar incluir las siguientes entidades:

- **Cliente**: Representa a las personas que utilizan los servicios de la agencia de viajes. Podría tener atributos como el nombre, dirección, correo electrónico, número de teléfono, historial de compras, etc.
- **Paquete de viaje:** Representa un conjunto de servicios ofrecidos por la agencia de viajes para un destino específico. Podría tener atributos como el destino, la duración, el precio, la descripción, etc.
- **Reserva**: Representa la reserva de un paquete de viaje por parte de un cliente. Podría tener atributos como el número de reserva, la fecha y hora de la reserva, el precio, el estado de la reserva, etc.
- **Vuelo**: Representa un vuelo específico incluido en un paquete de viaje. Podría tener atributos como el número de vuelo, la fecha y hora de salida y llegada, la compañía aérea, la clase, etc.

- **Hotel**: Representa un hotel específico incluido en un paquete de viaje. Podría tener atributos como el nombre del hotel, la ubicación, la categoría, el precio por noche, la disponibilidad de habitaciones, etc.

De estas entidades podrían derivarse algunas de las siguientes relaciones:

- Una reserva estaría relacionada con un cliente (1-1)
- Un paquete de viaje podría incluir varios vuelos y hoteles, y en los vuelos y hoteles se podrían alojar visitantes pertenecientes a distintos paquetes de viaje (N-N), (N-N)
- Etc.

2.5.3 Ejemplo de una universidad

En una universidad, y de una manera bastante simplificada, podríamos decidir incorporar las siguientes entidades:

- **Estudiante**: Representa a los alumnos que estudian en la universidad. Podría tener atributos como el nombre, dirección, correo electrónico, número de teléfono, historial académico, etc.
- **Profesor/a**: Representa a los/las docentes que imparten clases en la universidad. Podría tener atributos como el nombre, dirección, correo electrónico, número de teléfono, departamento, historial académico, etc.
- **Asignatura**: Representa una materia que se imparte en la universidad. Podría tener atributos como el código de asignatura, el nombre, la descripción, el número de créditos, etc.
- **Departamento**: Representa un departamento académico de la universidad. Podría tener atributos como el nombre del departamento, la descripción, la ubicación, el jefe del departamento, etc.
- **Matrícula**: Representa la matrícula de un estudiante en una asignatura específica. Podría tener atributos como el número de matrícula, la fecha de matriculación, la calificación, etc.

Entre las relaciones que se podrían establecer estarían:

- Una matrícula estaría relacionada con un/a estudiante (1-1) y varias asignaturas (1-N)
- Una asignatura estaría relacionada con un departamento (1-1) y con varios profesores, que pueden dar distintas asignaturas (N-N).

- Un/a profesor/a estaría relacionado/a con un departamento (1-1).
- Un departamento podría incluir a varios/as profesores/as. (1-N).

2.5.4 Ejemplo de un taller mecánico

En un taller mecánico, algunas de las entidades que podrían aparecer son:

- **Cliente**: Representa a los clientes que llevan sus vehículos al taller. Podría tener atributos como el nombre, dirección, correo electrónico, número de teléfono, límite de crédito, etc.
- **Vehículo**: Representa un vehículo que se lleva al taller para ser reparado. Podría tener atributos como la marca, modelo, año, número de identificación del vehículo (bastidor), etc.
- **Reparación**: Representa una reparación específica que se realiza en un vehículo. Podría tener atributos como el número de orden de trabajo, la fecha de inicio y finalización, el costo, la descripción del trabajo realizado, etc.
- **Mecánico/a**: Representa a quienes trabajan en el taller. Podría tener atributos como el nombre, dirección, correo electrónico, número de teléfono, especialidad, sueldo, etc.
- **Material**: Representa una parte específica que se utiliza en una reparación. Podría tener atributos como el nombre, la descripción, el número de pieza, el precio, el código de proveedor, etc.
- **Proveedor**: Empresa que sirve las piezas que sustituyen a piezas obsoletas o rotas en un proceso de reparación. Sus atributos podrían ser el CIF, nombre, dirección, etc.

Entre las relaciones que intuimos estarían:

- Una reparación estaría relacionada con un cliente (1-1)
- Una reparación estaría relacionada con un vehículo (1-1)
- Una reparación estaría relacionada con un mecánico responsable (1-1) y varios materiales (1-N).
- Cada material estaría relacionado con uno o varios proveedores, pudiendo estos servir distintos tipos de materiales (N-N).
- Etc.

3. Entidades fuertes vs entidades débiles. Relaciones de dependencia en existencia y en identificación

3.1 Entidad fuerte vs. Entidades débiles

El modelo Entidad-Relación distingue dos clases de conjuntos de datos en función de su autonomía semántica:

Una **entidad fuerte** representa un objeto que posee sentido completo por sí mismo. Sus ocurrencias se identifican mediante una clave primaria íntegra (por ejemplo, *ClienteID*, *HospitalID* o *ISBN*), de modo que ningún atributo externo resulta necesario para diferenciar cada registro. En el diagrama, la entidad fuerte se dibuja con un rectángulo de línea simple y su clave se subraya con trazo continuo.

Por el contrario, una **entidad débil** carece de identificador autosuficiente. El valor que la distingue —denominado **clave parcial**— sólo es único dentro del contexto de otra entidad, llamada **propietaria** u **originaria**. Para obtener un identificador completo se concatena la clave parcial con la clave primaria de la entidad fuerte. El diagrama ER refleja esta dependencia mostrando la entidad débil con un rectángulo de doble línea; la relación de identificación que la une a su propietaria se representa mediante un rombo también doble, y la participación del lado débil es necesariamente total, ya que ningún registro puede existir sin su dueño.

Aspecto comparativo	Entidad fuerte	Entidad débil
Identificador lógico	Clave primaria propia	Clave parcial + clave de la entidad fuerte
Existencia	Independiente	Dependiente de la entidad fuerte
Participación en la relación de identificación	Opcional o total, según la regla de negocio	Siempre total
Símbolo en el diagrama	Rectángulo de línea simple	Rectángulo de línea doble

La notación gráfica utilizada por la mayoría de herramientas de modelado ER se resume del modo siguiente:

Rectángulo de línea simple → entidad fuerte.

Rectángulo de doble línea → entidad débil.

Rombo de doble línea → relación de identificación entre la entidad fuerte y la débil.

Subrayado discontinuo en un atributo → clave parcial perteneciente a la entidad débil.

Criterios prácticos para decidir cuándo modelar una entidad como débil:

1. **Unicidad condicionada al propietario**
   Cuando el identificador "natural" resulta repetible entre distintos propietarios (por ejemplo, el número de asiento *12* aparece en cada avión) y se prefiere mantenerlo frente a introducir un código artificial, la entidad se declara débil y se añade la clave de la entidad fuerte para obtener unicidad global.
2. **Reforzamiento de la integridad referencial**
   La dependencia explícita simplifica la aplicación de reglas ON DELETE CASCADE u ON UPDATE CASCADE, evitando registros huérfanos y manteniendo el modelo alineado con el comportamiento real del negocio.
3. **Ciclo de vida subordinado**
   Una entidad que nunca debe permanecer en la base de datos después de eliminar a su propietaria se presta al patrón de entidad débil; la coexistencia de claves refleja y refuerza esa regla de permanencia.

Antes de adoptar este esquema conviene revisar si la solución alternativa —crear un identificador sintético en la presunta entidad débil— podría ofrecer mayor flexibilidad futura o un rendimiento más adecuado. La decisión óptima depende tanto de los requisitos de la aplicación como de las políticas de integridad y mantenimiento de la base de datos.

Ejemplo detallado de entidad fuerte / entidad débil en FestivalFest

Elemento	Descripción funcional	Representación en el diagrama ER
CONCIERTO (entidad fuerte)	Cada registro describe una actuación programada dentro del festival. Posee significado independiente y puede crearse, modificarse o eliminarse sin necesidad de otros objetos.	Rectángulo de línea simple con la clave primaria subrayada.
• Clave primaria	ConciertoID — identificador único.	*ConciertoID*
• Atributos básicos	FechaHora, EscenarioID (FK), ArtistaID (FK).	Aparecen dentro del rectángulo.
ENTRADA (entidad débil)	Representa un billete asociado a un concierto concreto. Su unicidad solo está garantizada junto con el concierto al que pertenece.	Rectángulo de línea doble; el atributo parcial se subraya con línea discontinua.
• Clave parcial	NúmeroEntrada — distingue los billetes dentro de un mismo concierto.	_ _ *NúmeroEntrada* _ _
• Clave completa	(ConciertoID, NúmeroEntrada) — combinación de la clave primaria de CONCIERTO y la clave parcial.	Se indica debajo de la entidad débil.
• Atributos adicionales	Precio, Tipo, Estado.	Listados en el rectángulo doble.

Relación de identificación EMITE	Une CONCIERTO con ENTRADA. Participación total en el lado débil; cardinalidad 1 : N.	Rombo de línea doble con texto «emite», flecha 1→N hacia ENTRADA.

Lectura del modelo

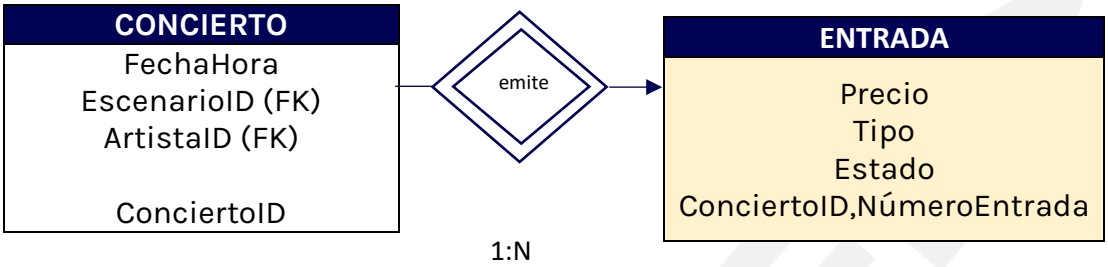
- Dependencia lógica
  - Una ENTRADA no puede existir sin su CONCIERTO asociado; la clave externa ConciertoID actúa como parte de su identificador.
  - CONCIERTO no necesita a ENTRADA para identificarse.
- Integridad referencial
  - La definición de la relación como 1 : N total autoriza cascadas de borrado (ON DELETE CASCADE). Si se cancela un concierto, el sistema puede retirar automáticamente todas las entradas emitidas para él, evitando registros «huérfanos».
- Racional de diseño
  - Se evita introducir una clave artificial en ENTRADA, pues el número de billete solo es único dentro de cada concierto.
  - La estructura refleja directamente el negocio del festival: los billetes carecen de sentido fuera del evento que los genera.

Símbolos que intervienen

Símbolo	Significado
Rectángulo simple	Entidad fuerte
Rectángulo doble	Entidad débil
Rombo doble	Relación de identificación
Subrayado continuo	Clave primaria
Subrayado discontinuo	Clave parcial de entidad débil

Con esta configuración el modelo capta fielmente la jerarquía «Concierto ⇒ Entrada», garantiza la consistencia de los datos y mantiene un esquema claro y ajustado a las reglas de negocio de FestivalFest.

La representación gráfica es la siguiente:



3.2 Relaciones de dependencia en existencia de identificación

En el modelo ER las relaciones de dependencia pueden ser de dos tipos:

Definición

Relación de existencia: La existencia de una entidad dependiente está condicionada por la existencia de la entidad de la que depende.

Las relaciones de dependencia describen cómo la vida de una entidad (hija) queda subordinada a otra (padre). En el modelo E/R se distinguen dos niveles de dependencia:

Tipo de dependencia	Idea esencial	Consecuencia sobre el diseño
Dependencia de existencia	Una fila hija no puede subsistir si la fila padre desaparece. El hijo *referencia* la clave primaria del padre mediante una clave foránea.	- El identificador del hijo no cambia: suele ser una clave propia (p. ej. PagoID). - La participación del hijo en la relación es total (doble línea o pie de cuervo con 0 prohibido). - Al eliminar la fila padre conviene propagar la acción (*ON DELETE CASCADE* o equivalente) para no dejar huérfanos.
Dependencia de identificación	Además de depender para existir, el hijo necesita la clave del padre para quedar totalmente identificado. El hijo es una *entidad débil*.	- El identificador completo se compone de clave parcial + clave del padre (p. ej. (PedidoID, NumeroLinea)). - Se representa con rectángulo doble y la relación con rombo doble. - La participación del hijo es total y la cardinalidad habitual es 1:N del padre al hijo.

Cuándo escoger cada una:

Situación	Diseñar como...	Motivo
El hijo posee un identificador natural único en toda la base pero no tiene sentido sin su padre (p. ej. *PagoID*).	Dependencia de existencia	Se mantiene una clave simple y la referencia bastará para la integridad.
El identificador natural del hijo es único solo dentro del ámbito del padre (p. ej. *Número de línea*).	Dependencia de identificación	Incluir la clave del padre evita claves artificiales y refleja la regla de negocio.

SI seguimos con el caso del festival de música, en el dominio de FestivalFest, la gestión de los pagos para la compra de entradas ilustra perfectamente una dependencia de existencia. A continuación, se describen las dos entidades implicadas, sus atributos y la naturaleza de su vínculo:

Entidad fuerte – **TICKET**

Ticket representa el billete comprado por un asistente para un concierto concreto. Puede existir de forma independiente mientras el concierto exista en el sistema.

Atributo	Descripción
TicketID	Clave primaria única de cada billete.
ConciertoID (FK)	Identificador del concierto al que da acceso.
AsistenteID (FK)	Identificador del asistente que adquiere el ticket.
Tipo	Categoría del billete (general, VIP, backstage…).
Precio	Importe pagado al adquirirlo.
Estado	'Emitido', 'Usado', 'Anulado', etc.

Entidad dependiente – **PAGO**

Pago recoge la transacción económica que valida la compra de uno o varios tickets. Dispone de su propia clave primaria, pero sin un ticket al que referenciar carece de sentido.

Atributo	Descripción
PaymentID	Clave primaria única para cada pago.
TicketID (FK)	Identificador del ticket cuya compra se está pagando.
FechaPago	Fecha y hora en que se efectuó el pago.
Importe	Importe abonado (coincidente con Precio del ticket).
MétodoPago	'Tarjeta', 'PayPal', 'Bizum', etc.

Naturaleza de la dependencia

La relación **efectúa** entre Ticket y Pago obedece a:

- **Cardinalidad 1:N**:

  Un único ticket genera (puede generar) varios registros de pago (por ejemplo, pago parcial, pago aplazado), pero cada pago se asocia a un único ticket.

- **Dependencia de existencia**:

  Ningún registro de Pago puede existir sin que antes exista su Ticket asociado. Si un ticket se elimina (por ejemplo, por devolución o anulación), todos los pagos vinculados deben borrarse automáticamente (ON DELETE CASCADE) para mantener la integridad del modelo.

- **Clave autónoma del hijo**:

  Aunque Pago dispone de su propia clave principal **PaymentID**, la validez de la transacción depende siempre de la existencia previa del Ticket referenciado.

Este diseño garantiza que:

1. **Integridad referencial**: no se registran abonos sobre tickets inexistentes.
2. **Acciones en cascada**: la eliminación de un ticket invalida y elimina sus pagos asociados sin necesidad de claves artificiales extra.
3. **Claridad semántica**: distingue claramente el billete (TICKET) de la operación económica (PAGO), reflejando la lógica real de FestivalFest.

En FestivalFest, la relación entre **PedidoComida** y **LíneaPedidoComida** ejemplifica una **dependencia de identificación**: la entidad hija no puede construirse ni distinguirse sin incorporar la clave de la entidad fuerte.

En detalle:

La entidad fuerte **PedidoComida** agrupa cada orden de un asistente en un puesto de comida. Dispone de:

- **PedidoID** como clave primaria, único en todo el sistema.
- **AsistenteID** (FK) para referenciar al asistente.
- **FechaHora** de la solicitud.
- **PuestoID** (FK) que indica el puesto reclamado.

La entidad débil **LíneaPedidoComida** representa cada renglón dentro de ese pedido. Carece de un identificador global propio; su atributo **NumeroLinea** sólo distingue las líneas dentro del mismo pedido. Por tanto:

- La clave completa se forma como **(PedidoID, NumeroLinea)**.

- **NumeroLinea** se subraya con línea discontinua (clave parcial).
- En el diagrama ER, se dibuja como rectángulo doble.

La **relación de identificación** "contiene":

- Se conecta con rombo doble entre **PedidoComida** y **LíneaPedidoComida**.
- Su cardinalidad es 1 Pedido → N Líneas, con participación total del lado débil (cada línea obligatoriamente pertenece a un pedido).
- Refleja que ninguna línea existe sin su pedido padre y que, para distinguir la línea 1 de un pedido 42 frente a la línea 1 de un pedido 73, es necesario conocer ambos componentes de la clave.

Este esquema cumple la **dependencia de identificación** porque:

1. **Identificación compuesta**: la línea sólo está unívocamente identificada al combinar **PedidoID** + **NumeroLinea**.
2. **Integridad referencial natural**: al eliminar un pedido, sus líneas desaparecen automáticamente (*ON DELETE CASCADE*).
3. **Ausencia de claves artificiales**: no se introduce un surrogate key adicional en **LíneaPedidoComida**, sino que se aprovecha la semántica propia del negocio.

Con ello, el modelo ER de FestivalFest refleja de forma fiel que las líneas de comanda son inseparables de su pedido original y necesitan su clave para definirse correctamente.

En el contexto de FestivalFest, la emisión y uso de bonos de acceso backstage ejemplifica una **relación de dependencia de identificación y existencia**.

En este modelo:

- Entidad fuerte: BonoBackstage
  - BonoID (PK
  - AsistenteID (FK)
  - ValidezDesde, ValidezHasta
- Entidad débil: AccesoBackstage
  - SecuenciaAcceso (clave parcial, subrayado discontinuo)

- Clave completa: (BonoID, SecuenciaAcceso)
- FechaHoraEntrada
- PuertaID (FK)

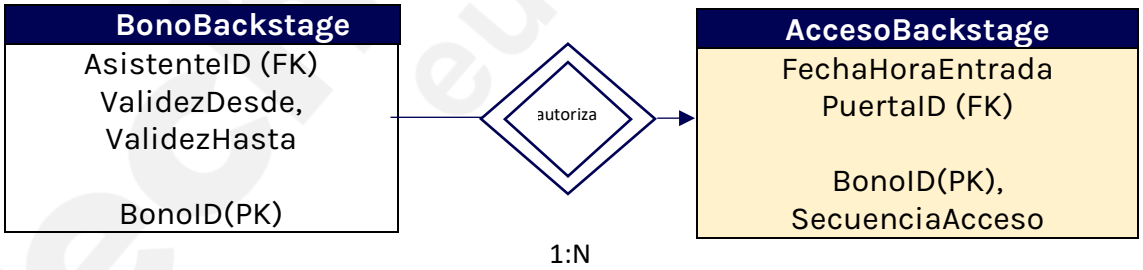
La relación **autoriza** entre BonoBackstage y AccesoBackstage presenta:

- **Cardinalidad 1:N**: un bono puede dar lugar a múltiples accesos, pero cada acceso pertenece a un único bono.
- **Dependencia de existencia**: ningún registro de AccesoBackstage tiene sentido si se elimina el BonoBackstage asociado (se implementa con ON DELETE CASCADE).
- **Dependencia de identificación**: SecuenciaAcceso solo distingue accesos dentro del mismo bono; para lograr unicidad se concatena con BonoID.

Diagrama ASCII simplificado:

BonoBackstage 1 ──♦♦ autoriza ── N AccesoBackstage

- Rectángulo simple para BonoBackstage (clave subrayada continua).
- Rectángulo doble para AccesoBackstage (clave parcial subrayada discontinua).
- Rombos dobles para la relación autoriza.



Este diseño cumple ambos requisitos:

1. **Identificación compuesta**: la entidad AccesoBackstage solo se identifica al combinar BonoID + SecuenciaAcceso.
2. **Existencia condicionada**: la existencia de cada acceso está ligada lógicamente a la subsistencia de su bono padre.

Así, el modelo ER refleja que cada uso de un bono backstage no solo depende de existir un bono válido, sino que además requiere incorporar su clave para distinguirlo de otros usos del mismo bono.

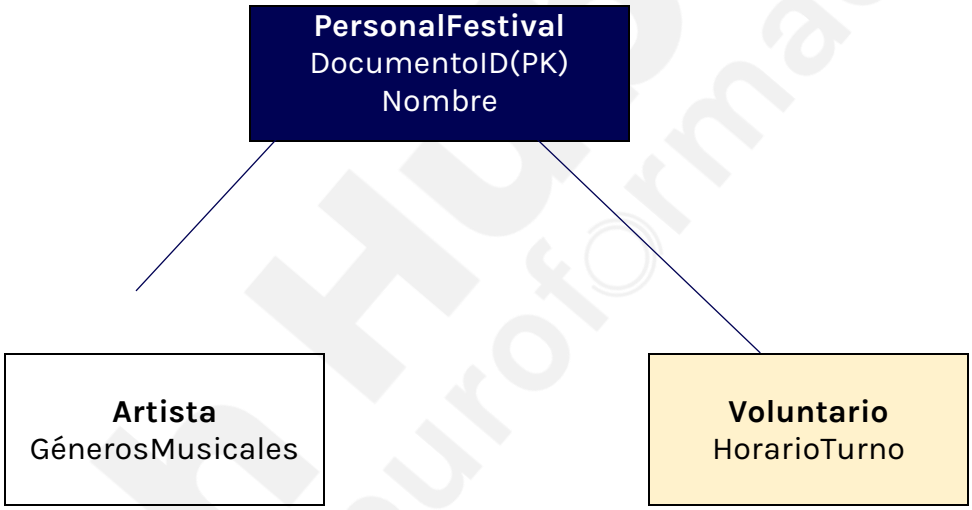
Tech Hub <FP> euroformac

4. El modelo E/R ampliado. Generalizaciones

4.1 El modelo E/R ampliado

El modelo Entidad-Relación ampliado (EER) toma la base del ER tradicional y la enriquece con mecanismos que permiten reflejar con mayor fidelidad las reglas de negocio y las jerarquías de un dominio complejo como FestivalFest.

En primer lugar, aparecen las **subentidades**, que heredan atributos y relaciones de una entidad padre pero añaden características propias. Por ejemplo, la entidad genérica "PersonalFestival" podría tener subentidades como "Artista" y "Voluntario". Ambas comparten campos como Nombre o DocumentoID, pero "Artista" incorpora además GénerosMusicales y "Voluntario" HorarioTurno. En el diagrama, "Artista" y "Voluntario" conectan a "PersonalFestival" mediante un triángulo de especialización o, en notación clásica, líneas punteadas hacia sus rectángulos hijos.



Importante

Las subentidades son entidades que están relacionadas con una entidad principal pero que representan una entidad separada.

La generalización y especialización se utilizan para crear una jerarquía de entidades que pueden compartir atributos y relaciones comunes, pero también tienen atributos y relaciones específicas para cada entidad en la jerarquía.

La **generalización** y su inversa, la **especialización**, formalizan estas jerarquías. Durante la generalización se agrupan entidades con rasgos comunes: si inicialmente existieran entidades "Seguridad", "Taquillero" y "Coordinador", puede definirse una única "PersonalFestival" que recoja sus atributos compartidos (Nombre, Turno) y luego se especialicen cada una manteniendo sólo aquello que las distingue (por ejemplo, "PermisosSeguridad" o "MáquinaVenta"). A la inversa, la especialización divide una entidad amplia en variantes más específicas, como transformar la entidad "Entrada" en "EntradaGeneral" y "EntradaVIP", cada una con sus propios campos (AccesoBackstage, PrecioPremium) además de heredar FechaEmisión o AsistenteID.

Las **relaciones recursivas** permiten modelar vínculos dentro de la misma entidad. En FestivalFest, un artista puede recomendar a otro, generando una relación "recomienda" que parte y llega a "Artista". Esto se dibuja uniendo el rectángulo de "Artista" consigo mismo mediante un rombo etiquetado y mostrando la cardinalidad adecuada (un artista puede hacer muchas recomendaciones, cada recomendación proviene de un único artista).

Importante

Las **relaciones recursivas** son aquellas en las que una entidad está relacionada consigo misma.

Por último, los **atributos multivaluados** reflejan propiedades que admiten múltiples valores simultáneamente. Si la entidad "Escenario" quiere registrar los géneros que acoge, se añade un atributo "Generos" que puede tomar varios valores (Rock, Electrónica, Indie). En la notación EER se representa con un óvalo doble unido a "Escenario".

Gracias a estas extensiones, el modelo EER de FestivalFest consigue:

- Reunir en una misma jerarquía ("PersonalFestival") funciones diversas manteniendo la posibilidad de especificar atributos únicos por rol.
- Simplificar el diagrama al evitar duplicar atributos comunes y, al mismo tiempo, preservar la individualidad de cada subentidad.
- Capturar recomendaciones internas entre artistas sin necesidad de crear entidades externas ad hoc.

- Reflejar naturalmente colecciones de valores (como los géneros de cada escenario) sin fragmentar la información en tablas auxiliares, reservando esta solución para la conversión al modelo relacional.

Importante

Los **atributos multivaluados** son aquellos que pueden tener más de un valor para una entidad.

En conjunto, el EER facilita el paso posterior a tablas relacionales, pues define con claridad las claves heredadas, las dependencias y las estructuras multivaluadas que luego se implementarán mediante relaciones y tablas intermedias.

4.2 Proceso de generalización

Definición

La **generalización** es el proceso de agrupar entidades que comparten características comunes en una entidad general

El proceso de generalización en el modelo Entidad-Relación ampliado consiste en identificar un conjunto de subentidades que comparten atributos y relaciones comunes, y elevar esos elementos compartidos a una única entidad padre más abstracta. De este modo, la entidad general recoge las propiedades y vínculos que resultan esenciales para todas las variantes, evitando la redundancia y favoreciendo la claridad del diseño.

En su aplicación práctica, la generalización se representa gráficamente uniendo cada subentidad a un triángulo de especialización invertido —o a un rombo con flechas— cuyos vértices convergen en la entidad padre. Esta convención indica que todas las instancias de las subentidades “heredan” automáticamente los atributos y las relaciones definidas en la entidad general.

Conceptualmente, la generalización permite trabajar con dos dimensiones complementarias:

- Reutilización de atributos y relaciones. En lugar de repetir en cada subentidad los mismos campos (por ejemplo, Identificador, Nombre, FechaRegistro) o vínculos (como "pertenece a"), esos elementos pasan a formar parte de la definición de la entidad padre. Así, cualquier cambio futuro en esas propiedades solo necesita aplicarse en un único lugar.
- Definición de jerarquías de tipo. La entidad general actúa como supertipo, mientras que las subentidades son los subtipos que, además de heredar lo común, pueden incorporar sus propias especializaciones. La semántica resultante refleja una relación "es-un" (IS-A) entre cada subentidad y su supertipo.

Desde el punto de vista de integridad, la generalización introduce dos restricciones opcionales que regulan la cobertura de la jerarquía:

- Completeness (totalidad o parcialidad): si la entidad padre debe representar todas las instancias de algún subgrupo (total) o permite elementos que no encajan en ninguna subentidad (parcial).
- Disjointness (exclusividad o solapamiento): si una instancia del supertipo puede pertenecer a una sola subentidad (disjunta) o a varias al mismo tiempo (solapada).

Gracias a este mecanismo, el modelo EER ofrece una abstracción poderosa para diseñar dominios complejos en los que distintos roles o categorías comparten rasgos comunes, asegurando un esquema más compacto y mantenible sin perder la capacidad de expresar diferencias específicas de cada subentidad.

4.3 Proceso de especialización

Definición

La **especialización** es el proceso opuesto a la generalización, en el que se dividen entidades en subentidades más específicas que comparten atributos y relaciones comunes.

El proceso de especialización invierte la lógica de la generalización: parte de una entidad genérica y la descompone en dos o más subentidades que capturan particularidades de grupos concretos dentro del conjunto original. En esta operación, la entidad padre conserva los atributos y relaciones comunes a todos los casos, mientras que cada subentidad añade los rasgos que la distinguen del resto.

Gráficamente, la especialización se representa mediante un triángulo o un rombo invertido conectado por líneas punteadas al rectángulo de la entidad padre y, desde él, a cada rectángulo que define la subentidad. Esta convención ilustra que cada instancia de la subentidad "hereda" automáticamente todos los atributos y vínculos del supertipo, y a la vez incorpora sus propias propiedades o asociaciones.

Existen dos criterios fundamentales que configuran la naturaleza de la especialización:

- Carácter de cobertura (completeness). Determina si todas las instancias de la entidad padre deben pertenecer necesariamente a alguna de las subentidades (especialización total) o si pueden existir ejemplos del supertipo que no encajen en ninguna de las categorías definidas (especialización parcial).
- Grado de exclusividad (disjointness). Establece si un objeto del supertipo puede clasificarse dentro de una sola subentidad (disjoint, garantizando que las subentidades son mutuamente excluyentes) o si puede simultáneamente pertenecer a varias de ellas (overlapping, admitiendo solapamientos de roles o propiedades).

La especialización refuerza la expresividad del modelo EER al permitir diseñar jerarquías ordenadas, evitar la repetición de atributos y relaciones, y mantener la coherencia semántica: cada subentidad representa un subconjunto claramente diferenciado, pero siempre dentro del marco conceptual compartido que define la entidad padre.

Caso práctico – Proceso de generalización en FestivalFest

Imagina que, como diseñador de la base de datos de FestivalFest, debes modelizar al personal que trabaja en el festival. Existen varios roles (Seguridad, Cajero y Voluntario) que comparten datos comunes (Nombre, DocumentoID, Turno), pero cada uno añade atributos específicos:

- Seguridad necesita registrar "PermisosEspeciales".
- Cajero requiere "MétodosPagoAceptados".

- Voluntario gestiona "ÁreaAsignada".

¿Cómo aplicarías el proceso de generalización para evitar repetir los campos comunes en cada rol y, al mismo tiempo, capturar las particularidades de cada uno?

Solución

1. Identificar los atributos y relaciones compartidas
   - Nombre
   - DocumentoID
   - Turno
     Cualquier cambio futuro en estos campos debe reflejarse en un único lugar.
2. Crear la entidad padre PersonalFestival

- Atributos:
  - PersonalID (clave primaria)
  - Nombre
  - DocumentoID
  - Turno

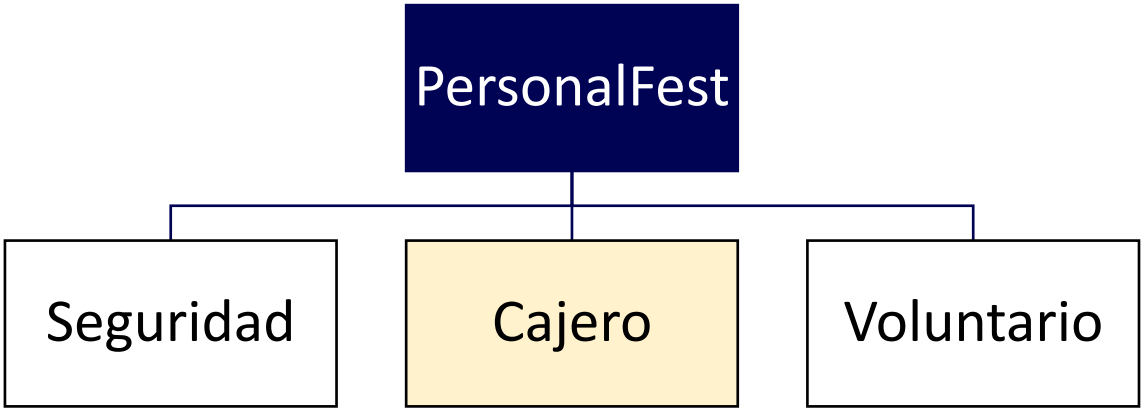
Representación ER: rectángulo simple "PersonalFestival" con PersonalID subrayado.

3. Definir las subentidades
   a. Seguridad–
      - Hereda Nombre, DocumentoID, Turno–
      - Añade PermisosEspeciales–
      - Se conecta a PersonalFestival con línea punteada y triángulo de generalización.
   b. Cajero–
      - Hereda Nombre, DocumentoID, Turno
      - Añade MétodosPagoAceptados

c. Voluntario– Hereda Nombre, DocumentoID, Turno
   - Añade ÁreaAsignada

4. Especificar restricciones de la jerarquía
   - Coverage (completeness): Parcial. Puede haber PersonalFestival de otros tipos no contemplados.
   - Disjointness (exclusividad): Disjunta. Un registro de PersonalFestival puede representar a un único rol.

Representación gráfica



- El triángulo invertido une PersonalFestival con cada subentidad.
- Las subentidades heredan los atributos del supertipo.

Gracias a esta generalización:

- Los atributos comunes se mantienen en un único sitio, simplificando el mantenimiento.
- Cada subentidad conserva sólo sus atributos propios, sin duplicar lo compartido.
- El modelo refleja claramente la relación IS-A entre PersonalFestival y cada rol.

5. Paso del modelo E/R al modelo relacional

Para **convertir un modelo entidad-relación en un modelo relacional**, se siguen los siguientes nueve pasos:

1. **Identificar las entidades y sus atributos: Se identifican todas las entidades en el modelo ER y se enumeran sus atributos.**

   En el primer paso, se identifican todas las entidades en el modelo ER y se enumeran sus atributos. Este proceso implica identificar los objetos, conceptos o cosas importantes en el sistema que se desea modelar y determinar sus características o propiedades.

   Algunos ejemplos de entidades en un modelo ER pueden incluir "Clientes", "Productos", "Pedidos", "Empleados", entre otros.

   Para cada entidad se enumeran sus atributos o características relevantes. Los atributos pueden ser de diferentes tipos como numéricos, de texto, de fecha, entre otros.

   Algunos ejemplos de atributos para la entidad "Clientes" pueden incluir "Nombre", "Apellido", "Correo electrónico", "Dirección", "Número de teléfono", entre otros.

   Es importante tener en cuenta que algunas entidades pueden tener atributos que son **claves primarias** o **claves foráneas**.

Definición

Las **claves primarias** son atributos que identifican de manera única cada registro en la tabla, mientras que las **claves foráneas** son atributos que hacen referencia a una clave primaria en otra tabla.

2. **Identificar las relaciones y sus cardinalidades: Se identifican las relaciones entre las entidades y se determinan las cardinalidades de cada relación.**

   En el segundo paso, se identifican las relaciones entre las entidades y se determinan las cardinalidades de cada relación.

Definición

Una **relación** es una asociación lógica entre dos o más entidades en el modelo ER. Las relaciones pueden ser de diferentes tipos como uno a uno, uno a muchos o muchos a muchos.

Para cada relación, es necesario identificar las entidades involucradas y determinar la cardinalidad de la relación, es decir, cuántos registros de una entidad están relacionados con cuántos registros de otra entidad.

Importante

La **cardinalidad** se representa mediante símbolos como "1" y "N", donde "1" representa un registro y "N" representa varios registros.

Por ejemplo, si tenemos una relación entre las entidades "Pedidos" y "Productos", donde un pedido puede contener varios productos y un producto puede estar en varios pedidos, la cardinalidad de la relación sería "N a N".

En cambio, si tuviéramos una relación entre las entidades "Clientes" y "Pedidos", donde un cliente puede hacer varios pedidos, pero cada pedido solo pertenece a un cliente, la cardinalidad de la relación sería "1 a N".

3. **Crear tablas para las entidades: Se crea una tabla para cada entidad en el modelo ER. El nombre de la tabla suele ser el mismo que el nombre de la entidad.**

   En el tercer paso se crean tablas para cada entidad en el modelo ER. Por ejemplo, si tenemos una entidad llamada "Clientes", el nombre de la tabla sería "Clientes".

   Cada tabla debe tener una clave primaria, que como sabemos, es un atributo o un conjunto de atributos que identifica de manera única cada registro en la tabla.

   Un ejemplo habitual es el que relaciona una tabla "Clientes" y una tabla "Pedidos": La clave primaria en la tabla "Clientes" se puede utilizar como clave foránea en la tabla "Pedidos" para vincular los pedidos con los clientes correspondientes.

En cada tabla se incluyen todos los atributos identificados para la entidad correspondiente en el paso anterior, incluyendo también los campos para las claves foráneas, si fuera necesario.

Importante

La clave primaria se utiliza para vincular registros de diferentes tablas en las relaciones.

Las claves foráneas se utilizan para vincular registros en diferentes tablas a través de las relaciones.

Es importante tener en cuenta que algunas entidades pueden tener relaciones de uno a muchos o muchos a muchos. En estos casos, se deben crear tablas adicionales para manejar estas relaciones. Por ejemplo, si tenemos una relación de uno a muchos entre las entidades "Clientes" y "Pedidos", se debe crear una tabla adicional para manejar la relación. Esta tabla adicional podría llamarse "Pedidos" y tendría una clave foránea que vincula cada pedido con el cliente correspondiente.

4. **Definir las columnas de la tabla: Cada columna de la tabla representa un atributo de la entidad. El nombre de la columna suele ser el mismo que el nombre del atributo.**

   En el cuarto paso se definen las columnas de la tabla para cada entidad.

Importante

Cada columna de la tabla representa un atributo de la entidad. El nombre de la columna suele ser el mismo que el nombre del atributo correspondiente.

Para cada columna es necesario especificar el tipo de datos que se almacenará en ella.

Por ejemplo, si tenemos un atributo "Nombre" en la entidad "Clientes", la columna correspondiente en la tabla "Clientes" tendría un tipo de datos **VARCHAR**, que se utiliza para almacenar cadenas de texto.

También es posible que se deban aplicar restricciones a las columnas para garantizar la integridad de los datos.

Por ejemplo, se puede aplicar una restricción de "NO NULL" a una columna para garantizar que siempre se ingrese un valor en ella.

Las claves primarias y las claves foráneas también deben representarse como columnas en las tablas correspondientes.

Las **claves primarias** deben ser únicas y no nulas, mientras que las **claves foráneas** deben hacer referencia a la clave primaria de la tabla relacionada, pudiendo ser nulas o no en función de si el valor de la columna va a ser obligatorio u opcional.

5. **Definir las claves primarias: Se define una clave primaria para cada tabla. La clave primaria es un atributo o una combinación de atributos que identifica de forma única cada fila en la tabla.**

   En el quinto paso se define una clave primaria para cada tabla.

Importante

La clave primaria se utiliza para garantizar la integridad de los datos y para vincular registros de diferentes tablas a través de las relaciones.

La clave primaria puede ser un atributo existente en la tabla o puede ser una combinación de varios atributos. **Si se utiliza una combinación de atributos se debe garantizar que la combinación sea única para cada fila en la tabla.**

Importante

Toda tabla debe tener una clave primaria.

Si la entidad en el modelo ER no tiene atributos únicos, se puede agregar un identificador único como clave primaria.

Este identificador puede ser un número de secuencia generado automáticamente por la base de datos.

6. **Crear tablas para las relaciones: Se crean tablas adicionales para representar las relaciones entre las entidades. El nombre de la tabla suele ser el mismo que el nombre de la relación.**

   En el sexto paso se crean tablas adicionales para representar las relaciones entre las entidades. El nombre de la tabla suele ser el mismo que el nombre de la relación.

Importante

Cada tabla de relación debe contener las claves primarias de las tablas relacionadas como claves foráneas.

Además, si la relación tiene atributos, se deben agregar columnas a la tabla de relación para representar estos atributos.

Por ejemplo, si tenemos una relación "Compra" entre las entidades "Clientes" y "Productos", se crearía una tabla "Compra" con columnas que representen las claves foráneas de las tablas "Clientes" y "Productos".

Si la relación tiene un atributo adicional, como "Fecha de compra", se agregaría una columna para este atributo en la tabla "Compra".

Importante

Las claves foráneas en la tabla de relación deben hacer referencia a las claves primarias en las tablas relacionadas.

Esto asegura que la integridad de los datos se mantenga al vincular registros de diferentes tablas a través de las relaciones.

7. **Definir las columnas de la tabla de relación: Cada columna de la tabla de relación representa una clave foránea que se refiere a la clave primaria de una de las tablas de entidad relacionada.**

   En el séptimo paso se definen las columnas de la tabla de relación.

Importante

Cada columna de la tabla de relación representa una clave foránea que se refiere a la clave primaria de una de las tablas de entidad relacionada.

Por ejemplo, si tenemos una tabla "Compra" que representa una relación entre las entidades "Clientes" y "Productos", la tabla "Compra" tendría dos columnas que representan las claves foráneas de las tablas "Clientes" y "Productos".

Estas columnas se utilizan para vincular los registros de la tabla "Compra" con los registros correspondientes en las tablas "Clientes" y "Productos".

Más info

Las **claves foráneas** en la tabla de relación deben hacer referencia a las claves primarias en las tablas relacionadas.

Esto asegura que la integridad de los datos se mantenga al vincular registros de diferentes tablas a través de las relaciones.

Si la relación tiene atributos adicionales se deben agregar columnas a la tabla de relación para representar estos atributos.

Estas columnas pueden ser de cualquier tipo de datos y se pueden utilizar para almacenar información adicional sobre la relación.

8. **Establecer las relaciones entre las tablas: Las relaciones se establecen mediante la definición de claves foráneas en las tablas de relación que se refieren a las claves primarias de las tablas de entidad relacionadas.**

   En el octavo paso se establecen las relaciones entre las tablas.

   Las relaciones se establecen mediante la definición de claves foráneas en las tablas de relación que se refieren a las claves primarias de las tablas de entidad relacionadas.

9. **Normalizar las tablas: Se aplican las reglas de normalización para reducir la redundancia y mejorar la eficiencia del sistema.**

   El último paso es la normalización de las tablas.

Definición

La **normalización** es un proceso de diseño de bases de datos que ayuda a reducir la redundancia y mejorar la eficiencia del sistema al dividir las tablas grandes en tablas más pequeñas y relacionadas.

Existen diferentes niveles de normalización, desde la primera forma normal (**1NF**) hasta la forma normal de **Boyce-Codd (BCNF**) y más allá.

La forma normal más comúnmente utilizada es la **tercera forma normal (3NF),** que elimina la redundancia de datos a través de la eliminación de dependencias transitivas.

Para aplicar las reglas de normalización es necesario examinar cuidadosamente la estructura de cada tabla y determinar si se pueden dividir en tablas más pequeñas y relacionadas.

Por ejemplo, si una tabla contiene atributos que dependen de otros atributos, se puede dividir en dos o más tablas para eliminar la redundancia.

La normalización también implica la eliminación de atributos que no están directamente relacionados con la clave primaria de la tabla, ya que estos atributos pueden contribuir a la redundancia de datos.

Importante

La normalización no siempre será necesaria o adecuada para todas las tablas, ya que puede generar tablas adicionales y aumentar la complejidad del sistema.

Veamos ahora cómo es el paso del modelo E/R al modelo relacional en el ejemplo de FestivalFest

Eres el responsable de la base de datos de FestivalFest y acabas de completar el diagrama Entidad–Relación con todas sus entidades y relaciones. Ahora debes pasar ese diseño al modelo relacional, de modo que quede listo para implementarse en un SGBD.

Necesitas aplicar los nueve pasos del "Paso del modelo E/R al modelo relacional" para transformar el diagrama de FestivalFest en un conjunto de tablas relacionales.

Solución

1. Identificar entidades y atributos

Entidad	Atributos
Concierto	ConciertoID (PK), FechaHora, EscenarioID (FK), ArtistaID (FK)
Artista	ArtistaID (PK), Nombre, GéneroMusical
Escenario	EscenarioID (PK), NombreEscenario, Capacidad
Coordinador	CoordinadorID (PK), Nombre, Turno
Asistente	AsistenteID (PK), Nombre, Email
Entrada	ConciertoID (PK-parcial), NúmeroEntrada (PK-parcial), Precio, Tipo, Estado

2. Identificar relaciones y cardinalidades

Relación	Participantes	Cardinalidad
ofrece	Artista → Concierto	1 Artista : N Conciertos
gestiona	Coordinador ↔ Escenario	1:1
asiste	Concierto ↔ Asistente	N:M
emite	Concierto → Entrada (entidad débil)	1 Concierto : N Entradas

3. Crear tablas para las entidades

- CONCIERTO
- ARTISTA
- ESCENARIO
- COORDINADOR

- ASISTENTE
- ENTRADA (entidad débil de Concierto)

4. Definir columnas de cada tabla

- CONCIERTO(
  ConciertoID, FechaHora, EscenarioID, ArtistaID
)
- ARTISTA(
  ArtistaID, Nombre, GéneroMusical
)
- ESCENARIO(
  EscenarioID, NombreEscenario, Capacidad
)
- COORDINADOR(
  CoordinadorID, Nombre, Turno
)
- ASISTENTE(
  AsistenteID, Nombre, Email
)
- ENTRADA(
  ConciertoID, NúmeroEntrada, Precio, Tipo, Estado
)

5. Definir claves primarias

- CONCIERTO: PK = ConciertoID
- ARTISTA: PK = ArtistaID
- ESCENARIO: PK = EscenarioID
- COORDINADOR: PK = CoordinadorID
- ASISTENTE: PK = AsistenteID
- ENTRADA: PK compuesta = (ConciertoID, NúmeroEntrada)

6. Crear tablas para las relaciones N:M

- ASISTE(
  ConciertoID (PK-parcial, FK→CONCIERTO),
  AsistenteID (PK-parcial, FK→ASISTENTE)
)

7. Definir columnas de la tabla de relación

- ASISTE:
  - ConciertoID (fk)
  - AsistenteID (fk)
  - PK = (ConciertoID, AsistenteID)

8. Establecer claves foráneas

- CONCIERTO.EscenarioID → ESCENARIO.EscenarioID
- CONCIERTO.ArtistaID → ARTISTA.ArtistaID
- ENTRADA.ConciertoID → CONCIERTO.ConciertoID
- ASISTE.ConciertoID → CONCIERTO.ConciertoID
- ASISTE.AsistenteID → ASISTENTE.AsistenteID

9. Normalizar las tablas

- 1FN: Todas las columnas atómicas; no hay multivalores ni repetición de grupos.
- 2FN: Cada tabla con clave compuesta (ENTRADA, ASISTE) ya no contiene atributos parciales.
- 3FN: No existen dependencias transitivas: los atributos dependen únicamente de la clave primaria de su tabla.

Con esto, el modelo relacional resultante queda libre de redundancias y listo para implementarse en FestivalFest.

6. Normalización de modelos relacionales, formas normales y desnormalización

5.1 Normalización de los modelos relacionales

La normalización de modelos relacionales es un proceso que se utiliza para diseñar y estructurar correctamente una base de datos relacional.

El objetivo principal de la normalización es reducir la redundancia de datos y asegurarse de que los datos estén almacenados de manera eficiente y coherente.

Existen varias formas normales (NF) que se utilizan para normalizar una base de datos relacional:

- Primera Forma Normal (1FN)
- Segunda Forma Normal (2FN)
- Tercera Forma Normal (3FN)
- Cuarta Forma Normal (4FN)
- Quinta Forma Normal (5FN)
- Forma Normal de Boyce-Codd (FNBC)

Hay que tener en cuenta que la normalización no siempre será la mejor opción para todas las situaciones.

En algunos casos, la desnormalización puede ser una estrategia efectiva para mejorar el rendimiento de una base de datos, siendo necesario analizar cuidadosamente cada caso y elegir la mejor estrategia para su situación específica.

5.2 Las formas normales

Las formas normales son reglas que se utilizan en el diseño de bases de datos relacionales para asegurar que los datos estén almacenados de manera coherente y eficiente, existiendo varias formas normales (NF) que se utilizan en el proceso de normalización.

5.2.1 Primera Forma Normal (1FN)

Para que una tabla alcance la Primera Forma Normal debe cumplir dos condiciones fundamentales:

1. **Atributos atómicos**

   Cada columna almacena un único valor elemental. No se permiten listas, conjuntos ni valores repetidos dentro de una misma celda. Por ejemplo, una columna GénerosMusicales en una tabla de **Artista** debe dividirse en una tabla auxiliar Artista_Géneros si un artista interpreta varios estilos.

2. **Registros únicos**

   No existen dos filas cuyas claves primarias coincidan, de modo que cada tupla sea inequívocamente identificable.

Consecuencia práctica: antes de la 1FN, las celdas con múltiples valores obligan a complejas operaciones de búsqueda y actualización. Cumplir la 1FN simplifica las consultas y garantiza que cada valor se trabaje de forma aislada.

5.2.2 Segunda Forma Normal (2FN)

Una tabla en 2FN debe estar ya en 1FN y, además:

- **Eliminar dependencias parciales**

  Si la clave primaria está formada por varios atributos (clave compuesta), ningún atributo no clave puede depender solo de una parte de esa clave.

Por ejemplo, en la tabla **Entrada** con clave (ConciertoID, NúmeroEntrada), un atributo como Tipo que dependa únicamente de NúmeroEntrada (y no de todo el par) debería moverse a una tabla TipoEntrada, dejando en **Entrada** solo aquellos atributos que verdaderamente dependan de ambos componentes de la clave.

Consecuencia práctica: la 2FN evita que al modificar una parte de la clave compuesta haya que actualizar atributos que no guardan relación con el conjunto completo, reduciendo anomalías de actualización y eliminando redundancias.

5.2.3 Tercera Forma Normal (3FN)

Partiendo de una tabla ya en 2FN, la 3FN exige:

- **Eliminar dependencias transitivas**

  Un atributo no clave no puede depender de otro atributo no clave. Cualquier dependencia indirecta con la clave debe resolverse separando los datos.

Por ejemplo, si en la tabla **Escenario** aparece la columna Ciudad, y existe además Provincia que depende de Ciudad, debe crearse una tabla **Ciudad** con su provincia y dejar en **Escenario** solo la referencia a CiudadID.

Consecuencia práctica: la 3FN asegura que cada columna aporte información directamente relacionada con la clave primaria, evitando que cambios en atributos ajenos a la clave generen inconsistencias.

5.2.4 Forma Normal de Boyce-Codd (FNBC)

La FNBC es un refuerzo de la 3FN que se aplica cuando:

- **Dependencias funcionales no triviales**

  Cualquier dependencia funcional en la tabla debe tener como determinante una clave candidata completa. Si un campo A determina B, entonces A debe ser, o parte de, una clave primaria o candidata.

En FestivalFest, si existiera una relación "Cada AsistenteID determina un GeneroPreferido", y AsistenteID no fuera clave candidata de la tabla **Asistente_Gustos**, habría que descomponer esa tabla para cumplir FNBC.

Consecuencia práctica: la FNBC impide escenarios en los que atributos aparentemente independientes sigan generando redundancia pese a estar en 3FN.

5.2.5 Cuarta Forma Normal (4FN)

Una vez en FNBC, para alcanzar la 4FN debe eliminarse:

- **Dependencias multivaluadas**

  Cuando un atributo A puede tener múltiples valores independientes de otro atributo B (por ejemplo, un **Escenario** con varios géneros musicales y a la vez varios patrocinadores), esas multivaluaciones deben separarse en tablas distintas (Escenario_Generos, Escenario_Patrocinadores).

Consecuencia práctica: la 4FN reduce redundancia cuando coexisten dos o más características que varían independientemente, evitando combinaciones explosivas de filas.

5.2.6 Quinta Forma Normal (5FN)

Partiendo de la 4FN, la 5FN exige:

- **Descomposición sin pérdida**

  Cualquier unión natural (JOIN) de tablas que descompone una relación compleja debe ser invertible sin generar ni duplicar ni perder información.

Esto suele surgir en relaciones ternarias o más: si tuviéramos una tabla que relacione Artista, Escenario y Turno, su descomposición en pares (Artista_Turno, Escenario_Turno) solo estaría en 5FN si con esas dos tablas se puede reconstruir exactamente la original.

Consecuencia práctica: la 5FN es rara en diseños diarios, pero garantiza que las descomposiciones complejas no alteren los conjuntos de datos al recombinarse.

Importante

No siempre es necesario o apropiado normalizar una base de datos hasta la 5FN.

El nivel de normalización adecuado dependerá de las necesidades específicas de la aplicación y la base de datos.

Incluso en algunos casos puede ser necesario desnormalizar una base de datos para mejorar el rendimiento o la eficiencia.

5.3 Ejemplos de normalización

Supongamos que estamos diseñando una base de datos para una tienda en línea que vende productos de electrónica. A continuación, se muestra un ejemplo de cómo se podría normalizar una tabla de productos para cumplir con las formas normales:

Primera Forma Normal (1FN):

En la tabla de productos, cada registro debe ser único y cada atributo debe ser atómico. Podríamos crear una tabla como la siguiente:

Código de producto	Nombre de producto	Categoría	Precio
1	Laptop Lenovo	Laptops	700
2	Smart TV Samsung	Televisores	1200
3	Mouse Inalámbrico HP	Accesorios	25

Segunda Forma Normal (2FN):

Para que una tabla esté en 2FN deben cumplirse dos condiciones:

1. Estar en 1FN (todos los atributos atómicos y clave única).
2. **Ningún atributo no clave** debe depender **parcialmente** de la clave primaria.

Tabla original (en 1FN)

CódigoProducto (PK)	Nombre	Categoría	Precio
1	Laptop Lenovo	Laptops	700
2	Smart TV Samsung	Televisores	1200
3	Mouse Inalámbrico HP	Accesorios	25

- **Clave primaria**: CódigoProducto.
- **Dependencia parcial**: el atributo Categoría no depende de **toda** la clave (que es sólo CódigoProducto), sino de una parte "categoría", por lo que viola 2FN.

Descomposición para 2FN

1. **Tabla Productos**: elimina Categoría y en su lugar añade CategoriaID como FK.
2. **Tabla Categorías**: nueva tabla que almacena cada categoría junto a su clave propia.

Tabla: Productos

CódigoProducto (PK)	Nombre	Precio	CategoriaID (FK)
1	Laptop Lenovo	700	1
2	Smart TV Samsung	1200	2
3	Mouse Inalámbrico HP	25	3

- **Clave primaria**: CódigoProducto.
- **Clave foránea**: CategoriaID → Categorías(CategoriaID).
- Ahora los atributos Nombre y Precio dependen **únicamente** de la clave completa CódigoProducto.

Tabla: Categorías

CategoriaID (PK)	NombreCategoría
1	Laptops
2	Televisores
3	Accesorios

- **Clave primaria**: CategoriaID.
- El atributo NombreCategoría depende **únicamente** de su propia clave primaria.

Con esta descomposición, cada tabla cumple 2FN:

Productos: todos sus campos no clave (Nombre, Precio, CategoriaID) dependen de la clave completa CódigoProducto.

Categorías: su único atributo no clave (NombreCategoría) depende de la clave completa CategoriaID.

Tercera Forma Normal (3FN)

Para que una tabla esté en 3FN debe cumplir 2FN y, además, **ningún atributo no clave dependa transitivamente** de la clave primaria.

En nuestro caso, tras aplicar 2FN, quedó:

Tabla Productos

CódigoProducto (PK)	Nombre	Precio	CategoriaID (FK)
1	Laptop Lenovo	700	1
2	Smart TV Samsung	1200	2
3	Mouse HP Inalámbrico	25	3

Tabla Categorías

CategoriaID (PK)	NombreCategoría
1	Laptops
2	Televisores
3	Accesorios

- En **Productos**, todos sus atributos (Nombre, Precio, CategoriaID) dependen **directamente** de la clave primaria CódigoProducto.
- En **Categorías**, NombreCategoría depende **directamente** de su clave primaria CategoriaID.
- No existe atributo en **Productos** que dependa de otro atributo no clave (por ejemplo, no aparece aquí NombreCategoría), por lo que **se elimina cualquier dependencia transitiva**.

Por tanto, esta estructura cumple **3FN** sin necesidad de una tabla intermedia adicional.

Cuarta Forma Normal (4FN)

Tabla Productos

CódigoProducto (PK)	Nombre	Precio	CategoriaID (FK)
1	Laptop Lenovo	700	1
2	Smart TV Samsung	1200	2
3	Mouse Inalámbrico HP	25	3

Tabla Categorías

CategoriaID (PK)	NombreCategoría
1	Laptops
2	Televisores
3	Accesorios

Aquí CategoriaID en Productos es una FK monovalor, y no hace falta una tabla intermedia.

Para la separación de un atributo multivalor ("Marcas"):

Tabla Marcas

MarcaID (PK)	NombreMarca
1	Lenovo
2	Samsung
3	HP

Tabla Productos-Marcas

CódigoProducto (FK)	MarcaID (FK)
1	1
1	3
2	2
3	3

- **Productos–Marcas** elimina la dependencia multivalor "un producto → varias marcas".
- Cada tabla de relación sólo contiene FKs y cumple 4FN (no hay dependencias no triviales ni múltiples valores en una misma columna).

Así se logra:

1. **1FN–3FN:** Productos con su FK monovalor a Categorías.
2. **4FN:** Marcas extraídas a su propia tabla, con intermedia Productos–Marcas.

De este modo no se crean enlaces innecesarios (Productos-Categorías) y se separan propiamente sólo los multivalores (Marcas).

Quinta Forma Normal (5FN)

Una tienda on-line de electrónica ofrece sus artículos a través de distintos distribuidores y en varias **regiones**.

Una *oferta* solo existe cuando se fijan **las tres dimensiones al mismo tiempo**:

- **Producto** (ProductoID)
- **Distribuidor** (DistribuidorID)
- **Región** (RegionID)

1. Relación triádica inicial

Tabla **Oferta** (ya en 4FN)

ProductoID	DistribuidorID	RegionID	Precio
P1	D1	R1	699
P1	D1	R2	719
P2	D2	R1	1199

Clave primaria: **(ProductoID, DistribuidorID, RegionID)**.

No hay dependencias funcionales ni de valores múltiples, pero **aún no estamos en 5FN**.

2. Descomposición en proyecciones binarias

Tabla	Clave primaria
Prod-Dist	(ProductoID, DistribuidorID)
Prod-Reg	(ProductoID, RegionID)
Dist-Reg	(DistribuidorID, RegionID)

Contenido mínimo para conservar toda la información

Prod-Dist

ProductoID	DistribuidorID
P1	D1
P2	D2

Prod-Reg

ProductoID	RegionID
P1	R1
P1	R2
P2	R1

Dist-Reg

DistribuidorID	RegionID
D1	R1
D1	R2
D2	R1

3. Verificación de *join dependency*

Al reunir las tres proyecciones:

```
SELECT pd.ProductoID,
      pd.DistribuidorID,
      pr.RegionID
 FROM ProdDist pd
 JOIN DistReg dr
       ON pd.DistribuidorID = dr.DistribuidorID
 JOIN ProdReg pr
       ON pr.ProductoID = pd.ProductoID
      AND pr.RegionID   = dr.RegionID;
```

se reconstruyen **exactamente** las tuplas originales de **Oferta** y **no aparece ninguna combinación espuria**.

La relación triádica depende únicamente de la unión de sus proyecciones binarias ⇒ **5ª Forma Normal**.

4. Propiedades conseguidas

- **Ausencia de redundancia**: la misma pareja (por ejemplo, *P1–D1*) se guarda una sola vez.
- **Integridad**: ninguna inserción en las tablas binarias puede generar ofertas imposibles; la unión reproduce solo las válidas.
- **Flexibilidad**: si un nuevo distribuidor sirve una región, basta una fila en **Dist-Reg** para que las ofertas correctas puedan generarse al unir las tablas.

Este es el patrón típico de 5FN: una relación n-aria ($n \geq 3$) cuyos datos **solo pueden derivarse de la unión de todas sus proyecciones binarias**, garantizando irredundancia y consistencia sin dependencias funcionales residuales.

Forma Normal de Boyce-Codd (FNBC):

Supongamos que algunos clientes pueden tener varias direcciones de envío, y que una dirección de envío solo se aplica a un cliente. Para evitar redundancia y posibles inconsistencias, podríamos crear dos tablas separadas:

Tabla Clientes:

Código cliente	Nombre del cliente
1	Juan Pérez
2	María Rodríguez
3	Pedro Gómez

Tabla Direcciones:

Código de cliente	Código de dirección	Dirección
1	1	Calle 1 #123, Ciudad A
1	2	Calle 2 #546, Ciudad A

2	3	Calle 3 #789, Ciudad B
3	4	Calle 4 #1011, Ciudad C

La tabla Direcciones tiene una clave principal compuesta por las claves de Cliente y Dirección, y ambas columnas son claves foráneas en la tabla de Clientes.

Esto significa que la tabla Direcciones no tiene dependencias funcionales no triviales, ya que la clave principal es la mínima para identificar de forma única cada fila.

Al separar las direcciones de los clientes en su propia tabla, evitamos la redundancia de datos y eliminamos la posibilidad de inconsistencias en los datos.

Con este diseño se cumple la **FNBC** y se asegura que cada tabla tenga una única clave principal y no tenga dependencias funcionales no triviales.

5.4 Desnormalización

La **desnormalización** es un proceso que se utiliza para aumentar el rendimiento de una base de datos al reducir el número de tablas y simplificar las consultas, a costa de aumentar la redundancia de los datos y posiblemente perder alguna integridad referencial.

Generalmente se realiza en las tablas más utilizadas o en aquellas que tienen un mayor impacto en el rendimiento de la base de datos.

El proceso de desnormalización implica agregar redundancia de datos en una o varias tablas para evitar unir varias tablas y mejorar el rendimiento de las consultas, lo que se puede lograr de varias maneras, como se describe a continuación:

- **Combinación de tablas**: En lugar de tener varias tablas con relaciones complejas, se pueden combinar las tablas relacionadas en una sola tabla. Esto puede ayudar a evitar unir varias tablas y simplificar las consultas, aunque a costa de la redundancia de datos.
- **Creación de columnas derivadas:** Se pueden agregar columnas calculadas o derivadas a una tabla para evitar tener que calcularlas en cada consulta. Por ejemplo, en lugar de tener que calcular la suma de los valores de una columna en cada consulta, se puede agregar una columna con el total.

- **Creación de índices**: La creación de índices en las columnas utilizadas con frecuencia en consultas puede mejorar el rendimiento de la base de datos, ya que permite que las consultas se ejecuten más rápidamente.
- **Eliminación de claves foráneas**: La eliminación de claves foráneas puede mejorar el rendimiento de la base de datos al evitar la necesidad de unir varias tablas en cada consulta. Sin embargo, esto también puede reducir la integridad referencial de la base de datos.

Importante

La desnormalización no siempre es la mejor opción y debe realizarse con precaución.

La redundancia de datos puede aumentar la posibilidad de inconsistencias y errores en los datos, y la eliminación de claves foráneas puede reducir la integridad referencial de la base de datos.

Por esta y otras razones, la desnormalización debe considerarse cuidadosamente en función de las necesidades específicas de la aplicación y del rendimiento de la base de datos.

Ejemplo de Desnormalización

Supongamos que tenemos una base de datos para una tienda en línea que vende productos electrónicos, con las siguientes tablas:

Tabla Productos:

Código de producto	Nombre de producto	Descripción	Precio

Tabla Clientes:

Código de cliente	Nombre del cliente	Dirección	Teléfono

Tabla Pedidos:

Código de pedido	Código del cliente	Código de producto	Cantidad

En este diseño hay una tabla de productos, una tabla de clientes y una tabla de pedidos que relaciona las dos primeras tablas a través de claves foráneas.

Sin embargo, cada vez que necesitamos obtener información sobre un pedido específico, tenemos que unir tres tablas, lo que puede ser costoso en términos de rendimiento.

Para mejorar el rendimiento de la base de datos, podríamos considerar la desnormalización, agregando redundancia de datos en la tabla de pedidos.

Por ejemplo, podríamos agregar las columnas Nombre de Producto y Precio a la tabla de pedidos, de la siguiente manera:

Tabla pedidos:

Código de pedido	Código de cliente	Código de producto	Cantidad	Nombre de producto	Precio

Con este nuevo diseño, cada vez que necesitamos obtener información sobre un pedido específico, solo tenemos que consultar la tabla de pedidos, que ahora contiene toda la información relevante en una sola tabla.

Aunque esto puede aumentar la redundancia de datos, también puede mejorar significativamente el rendimiento de las consultas.

6.Resumen

Entidades y relaciones. Atributos. Cardinalidad:

- Las entidades son objetos o conceptos en el mundo real que se modelan en una base de datos.
- Los atributos son características o propiedades de una entidad.
- Las relaciones son las conexiones entre dos o más entidades.
- La cardinalidad se refiere a la cantidad de instancias de una entidad que pueden estar relacionadas con instancias de otra entidad.

Entidades fuertes vs. entidades débiles. Relaciones de dependencia en existencia y en identificación:

- Las entidades fuertes tienen una clave primaria única que las identifica.
- Las entidades débiles no tienen una clave primaria única y dependen de una entidad fuerte para ser identificadas.
- La relación de dependencia en existencia se da cuando una entidad débil no puede existir sin la existencia de una entidad fuerte.
- La relación de dependencia en identificación se da cuando una entidad débil se identifica por la combinación de su clave y la clave primarias de su entidad fuerte asociada.

El modelo E/R ampliado. Generalizaciones:

- El modelo E/R ampliado es una extensión del modelo E/R básico que incluye conceptos adicionales, como la generalización y la especialización.
- La generalización se refiere al proceso de combinar varias entidades similares en una entidad más general.
- La especialización se refiere al proceso de dividir una entidad general en entidades más específicas.

Paso del modelo E/R al modelo relacional:

- El paso del modelo E/R al modelo relacional implica la conversión de entidades, atributos y relaciones en tablas, columnas y claves foráneas, respectivamente.
- Cada tabla representa una entidad o una relación, y cada columna representa un atributo.
- Las claves primarias y las claves foráneas se utilizan para establecer las relaciones entre las tablas.

Normalización de modelos relacionales. Formas normales. Desnormalización:

- La normalización es el proceso de organizar una base de datos para minimizar la redundancia y mejorar la integridad de los datos.
- Las formas normales son reglas que especifican ciertas condiciones que deben cumplirse para que una base de datos esté normalizada.
- Las formas normales más comunes son la primera forma normal (1FN), la segunda forma normal (2FN), la tercera forma normal (3FN), la forma normal de Boyce-Codd (FNBC) y la cuarta forma normal (4FN) y quinta forma normal (5FN).
- La desnormalización es un proceso que se utiliza para aumentar el rendimiento de una base de datos al reducir el número de tablas y simplificar las consultas, a costa de aumentar la redundancia de los datos y posiblemente perder alguna integridad referencial.

Glosario

Entidad: un objeto o concepto en el mundo real que se modela en una base de datos.

Atributo: una característica o propiedad de una entidad.

Relación: una conexión entre dos o más entidades.

Cardinalidad: la cantidad de instancias de una entidad que pueden estar relacionadas con instancias de otra entidad.

Entidad fuerte: una entidad que tiene una clave primaria única que la identifica.

Entidad débil: una entidad que no tiene una clave primaria única y depende de una entidad fuerte para ser identificada.

Relación de dependencia en existencia: una relación entre una entidad débil y una entidad fuerte en la que la entidad débil no puede existir sin la existencia de la entidad fuerte.

Relación de dependencia en identificación: una relación entre una entidad débil y una entidad fuerte en la que la entidad débil se identifica por la combinación de su clave y la clave primarias de su entidad fuerte asociada.

Modelo E/R ampliado: una extensión del modelo E/R básico que incluye conceptos adicionales, como la generalización y la especialización.

Generalización: el proceso de combinar varias entidades similares en una entidad más general.

Especialización: el proceso de dividir una entidad general en entidades más específicas.

Tabla: una representación de una entidad o relación en el modelo relacional.

Columna: una representación de un atributo en el modelo relacional.

Clave primaria: una columna o conjunto de columnas que identifican de forma única cada fila de una tabla.

Clave foránea: una columna o conjunto de columnas en una tabla que se refiere a la clave primaria de otra tabla.

Normalización: el proceso de organizar una base de datos para minimizar la redundancia y mejorar la integridad de los datos.

Formas normales: reglas que especifican ciertas condiciones que deben cumplirse para que una base de datos esté normalizada.

Primera forma normal (1FN): una tabla no debe contener grupos repetitivos de columnas.

Segunda forma normal (2FN): una tabla debe estar en 1FN y todas las columnas deben depender de la clave primaria.

Tercera forma normal (3FN): una tabla debe estar en 2FN y todas las columnas deben depender solo de la clave primaria.

Forma normal de Boyce-Codd (FNBC): una tabla debe estar en 3FN y cada dependencia funcional debe tener una clave candidata como clave primaria.

Cuarta forma normal (4FN): una tabla debe estar en 3FN y no debe haber dependencias multivaluadas o dependencias de join.

Quinta forma normal (5FN): una tabla debe estar en 4FN y no debe haber dependencias de unión o dependencias de join con más de una clave candidata.

Desnormalización: Es el proceso de agregar redundancia a un modelo de datos relacional para mejorar el rendimiento de consultas. En lugar de seguir reglas estrictas de normalización para eliminar la redundancia de datos, se agregan duplicados de datos a una o más tablas en el modelo de datos.

Redundancia: datos repetidos en varias tablas. La redundancia puede aumentar el rendimiento de las consultas, pero también aumenta la complejidad y el costo de mantenimiento.

Bibliografía

Barker, R. (1994). El modelo Entidad-Relación CASE*Method. Addison-Wesley.

Silberschatz, A., Korth, H., & Sudarshan, S. (2014). Fundamentos de bases de datos. McGraw-Hill.

Ramakrishnan, R., & Gehrke, J. (2005). Sistemas de bases de datos (4ta ed.). Addison-Wesley Iberoamericana.

Tech Hub <FP> euroformac